Quantization

Daniele Solombrino
GLADIA @ Sapienza University of Rome

Quantization
Part I

Daniele Solombrino
GLADIA @ Sapienza University of Rome

Dan

- Daniele Solombrino
- solombrino@di.uniroma1.it
- dansolombrino @ EVERYTHING (except TikTok… too old for that 👴)
- DMs open for info, collabs and… banger EDM songs!

Dan

PhD student in AI w/ GLADIA @ Sapienza University of Rome

- Tried to make models reusable → model merging
- Tried to make models controllable → model steering
- Tryin’ to make models efficient → model quantization

Dan

AI engineering

- (multimodal) LLMs for Italian museums, historical and cultural associations

Dan

AI teaching

- BSc and MSc ML/DL courses
- How companies can create and maintain AI products
- AI tools for culture workers and biologists

Dan

BSc and MSc in computer science

- High-performance computing
- Machine learning
- Deep learning

Background

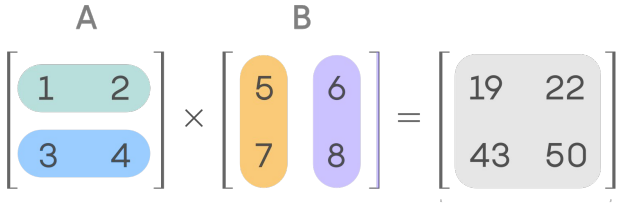
Matrix multiplication

```python
import torch
from torch.utils.data import DataLoader
from transformers import ViTForImageClassification
from datasets import load_dataset

# Data
dataset = load_dataset("cifar10")
loader = DataLoader(dataset, batch_size=8, shuffle=True)

# Model
model = ViTForImageClassification.from_pretrained(
    "google/vit-base-patch16-224",
    num_labels=10
)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Training
optimizer = torch.optim.AdamW(model.parameters(), lr=5e-5)
num_epochs = 5

for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0

    for batch in loader:
        inputs = batch["pixel_values"].to(device)
        targets = batch["label"].to(device)

        optimizer.zero_grad()
        outputs = model(pixel_values=inputs, labels=targets)
        outputs.loss.backward()
        optimizer.step()
        running_loss += outputs.loss.item()

    avg_loss = running_loss / len(loader)
    print(f"Epoch {epoch+1}/{num_epochs} — Loss: {avg_loss:.4f}")
```

Matrix multiplication

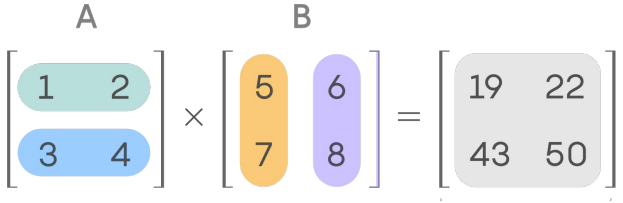
```python
import torch
from torch.utils.data import DataLoader
from transformers import ViTForImageClassification
from datasets import load_dataset

# Data
dataset = load_dataset("cifar10")
loader = DataLoader(dataset, batch_size=8, shuffle=True)

# Model
model = ViTForImageClassification.from_pretrained(
    "google/vit-base-patch16-224",
    num_labels=10
)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Training
optimizer = torch.optim.AdamW(model.parameters(), lr=5e-5)
num_epochs = 5

for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0

    for batch in loader:
        inputs = batch["pixel_values"].to(device)
        targets = batch["label"].to(device)

        optimizer.zero_grad()
        outputs = model(pixel_values=inputs, labels=targets)
        outputs.loss.backward()
        optimizer.step()
        running_loss += outputs.loss.item()

    avg_loss = running_loss / len(loader)
    print(f"Epoch {epoch+1}/{num_epochs} — Loss: {avg_loss:.4f}")
```

Matrix multiplication



Matrix multiplication

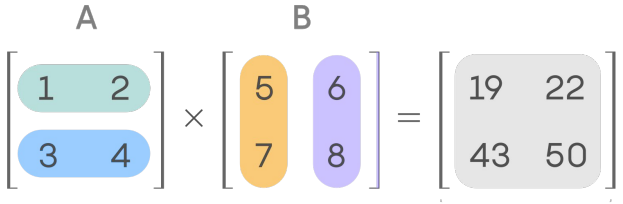
What math operations do we need for matmul?



Matrix multiplication

What math operations do we need for ~~matmul~~?

What math operations do we need for dot prod?

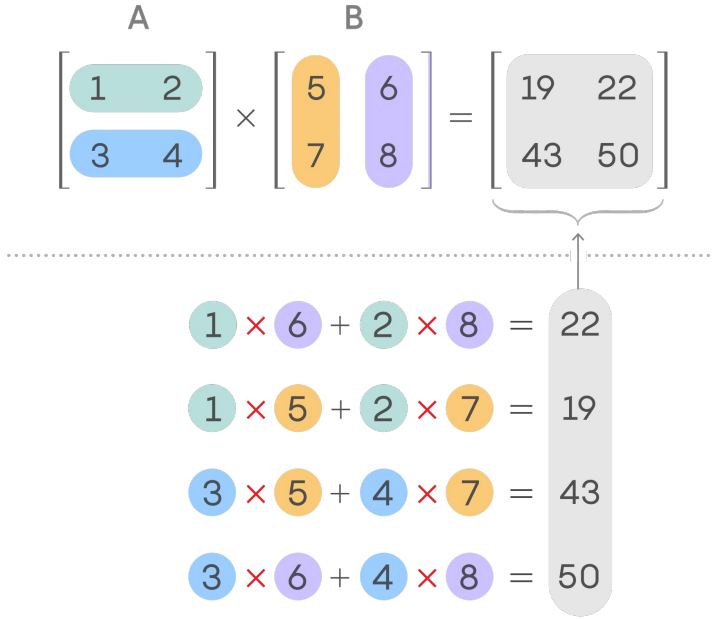


Matrix multiplication

What math operations do we need for ~~matmul~~?

What math operations do we need for dot prod?

Scalar multiplication && scalar addition



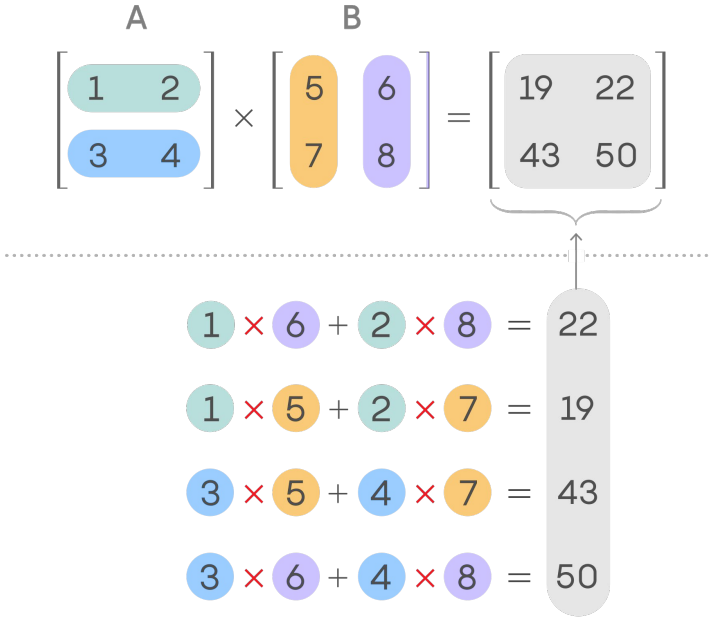
Matrix multiplication

What math operations do we need for ~~matmul~~?

What math operations do we need for dot prod?

Scalar multiplication && scalar addition

A.K.A. multiply and accumulate (MAC)



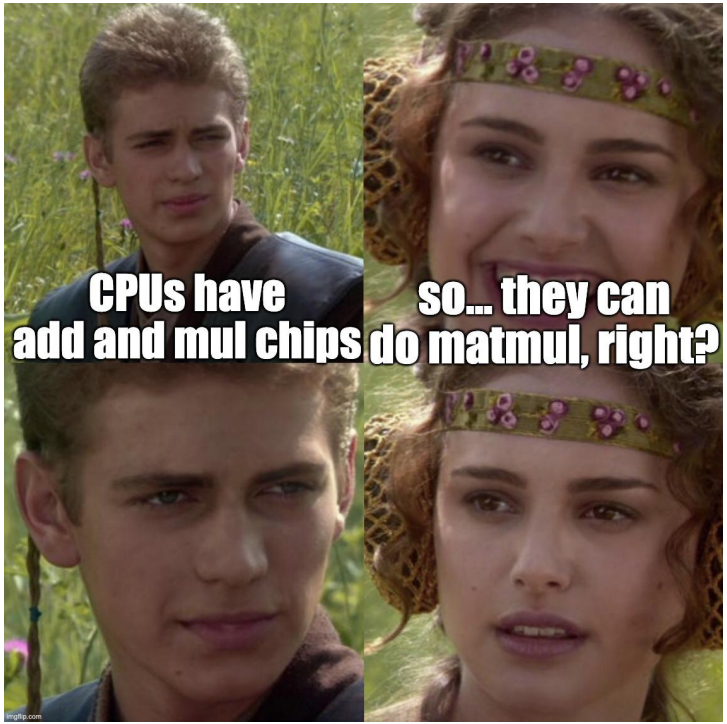
Matrix multiplication

What does it mean for a computer to be able to do an operation?

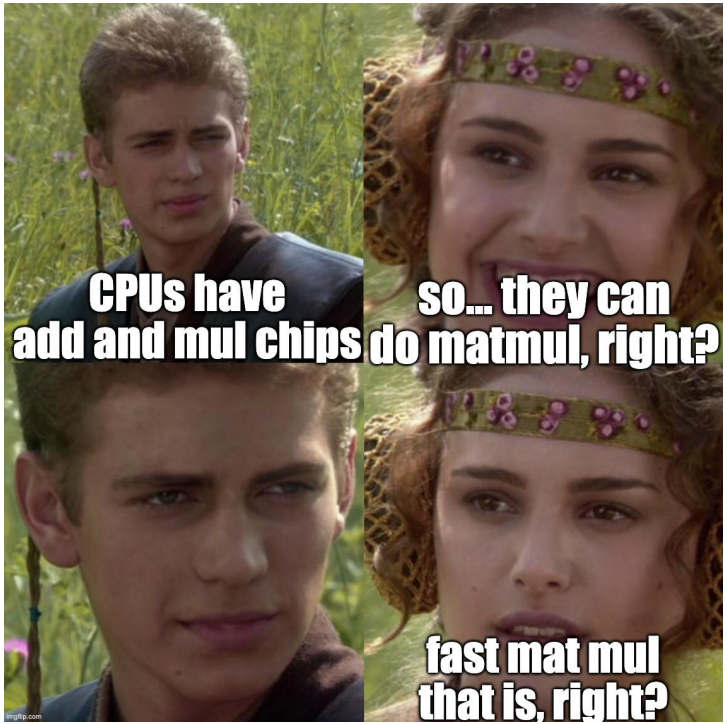
Its CPU must have hardware capable of carrying that op out

Fast matrix multiplication

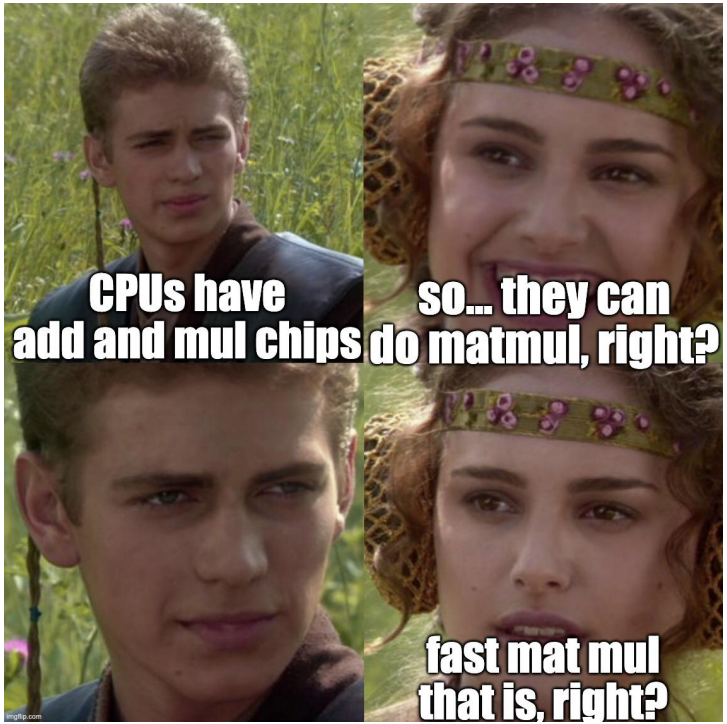
Fast matrix multiplication

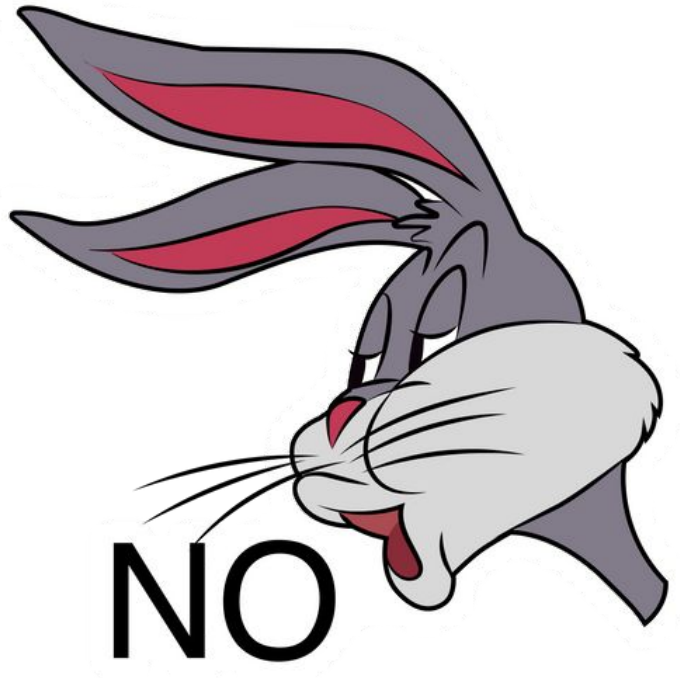


Fast matrix multiplication



Fast matrix multiplication





CPUs vs. GPUs

CPUs can do matmul… slow :(

CPUs vs. GPUs

CPUs can do matmul… slow :(

- A bunch of dot prods at the same time

CPUs vs. GPUs

CPUs can do matmul… slow :(

- A bunch of dot prods at the same time
- Dot prod treated as a sequence of MAC operations

CPUs vs. GPUs

GPUs can do matmul… fast :)

CPUs vs. GPUs

GPUs can do matmul… fast :)

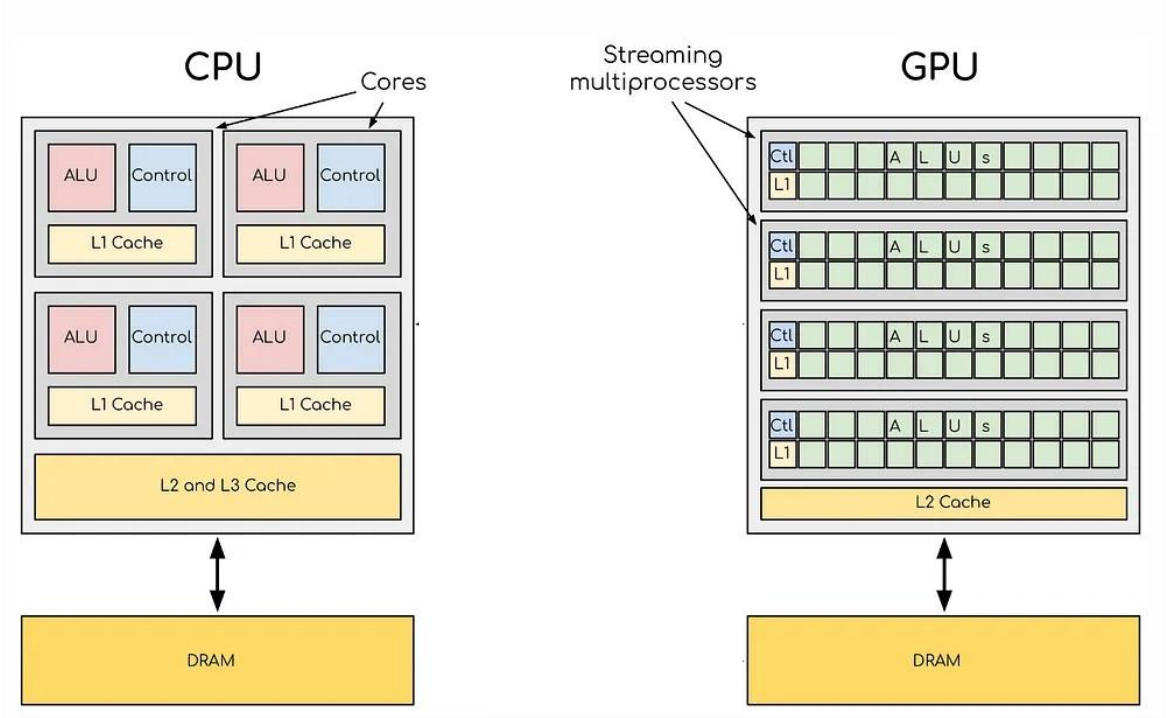
- A LOT of dot prods at the same time → CUDA cores (before 2017)

CPUs vs. GPUs

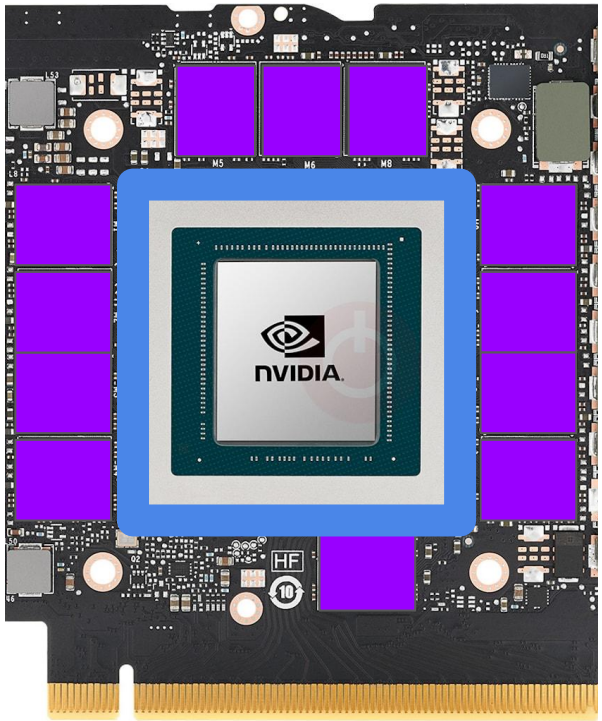
GPUs can do matmul… fast :)

- A LOT of dot prods at the same time → CUDA cores (before 2017)
- Dot prod treated as a single op → Tensor cores (from 2017)

CPUs vs. GPUs

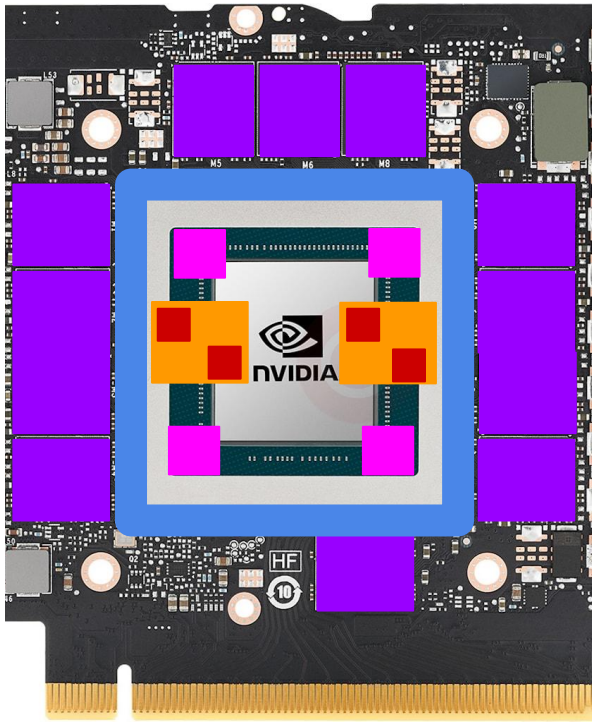

CPU
Cores
ALU
Control
L1 Cache
L2 and L3 Cache
DRAM
Streaming multiprocessors
GPU
Ctl
A L U s
L1
L2 Cache
DRAM

Motivation



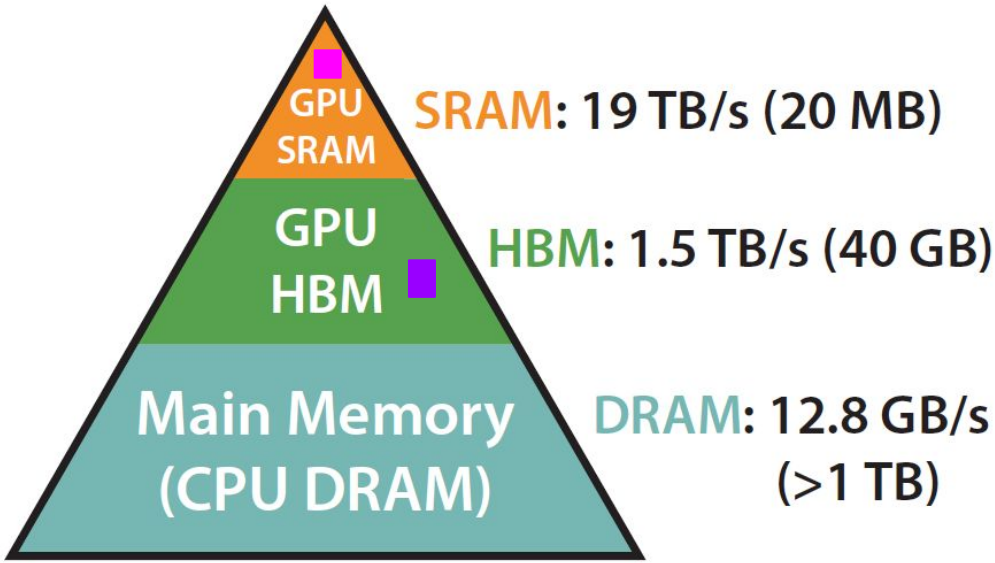
- **Memory**
- **GPU**

Motivation



- **Memory**
  - **Dynamic RAM (DRAM)**
- **GPU**
  - **Streaming Multiprocessor (SM)**
  - **Register**
  - **Static RAM (SRAM)**

GPUs memory hierarchy



GPU programming

Different arch → different programming style

A GPU program → “kernel”

kernel programming → next week!

Motivation

Motivation

torch.matmul() under the hood

torch.matmul(input, other)

torch.matmul() under the hood

torch.matmul(input, other)

↓

Kernel selection + start

torch.matmul() under the hood

torch.matmul(input, other)

↓

Kernel selection + start

↓

DRAM → SRAM/registers data move

torch.matmul() under the hood

torch.matmul(input, other)

↓

Kernel selection + start

↓

DRAM → **SRAM**/**registers** data move

↓

Mat mul

torch.matmul() under the hood

torch.matmul(input, other)

↓

Kernel selection + start

↓

DRAM → SRAM/registers data move

↓

Mat mul

↓

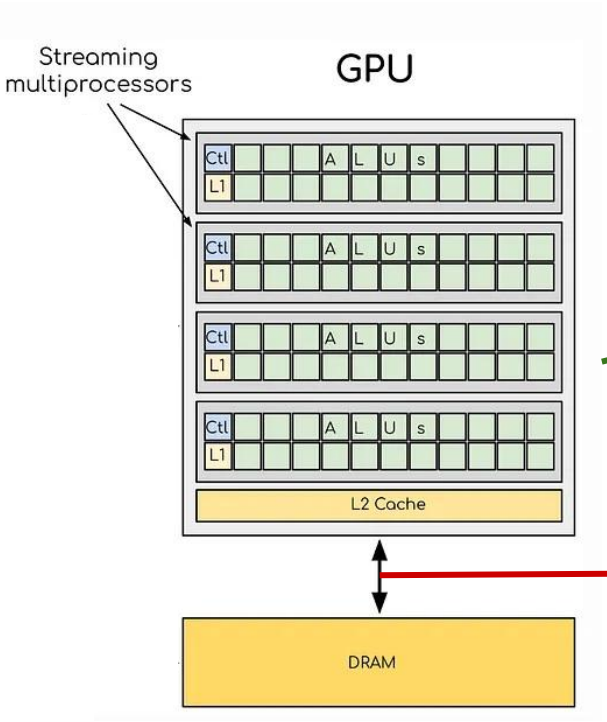
SRAM/registers → DRAM data move

Compute and memory cost

Compute ops → cheap

Data moves → (200x more) expensive

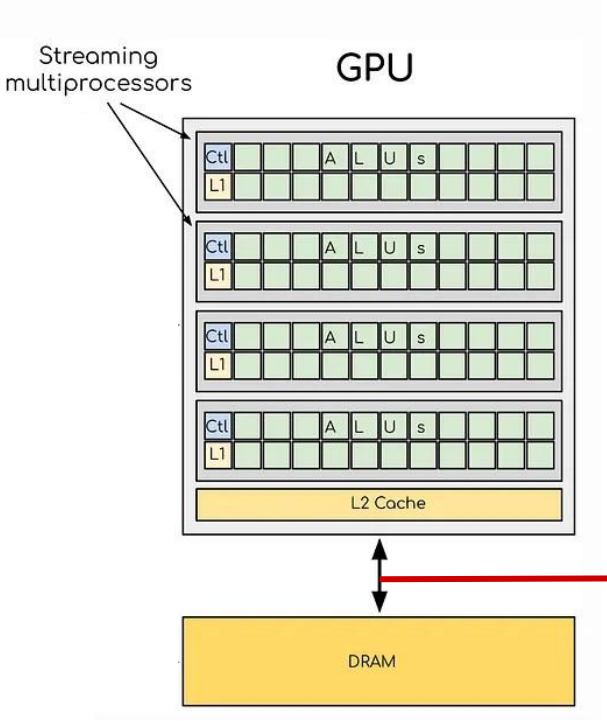
Compute and memory cost



Cheap

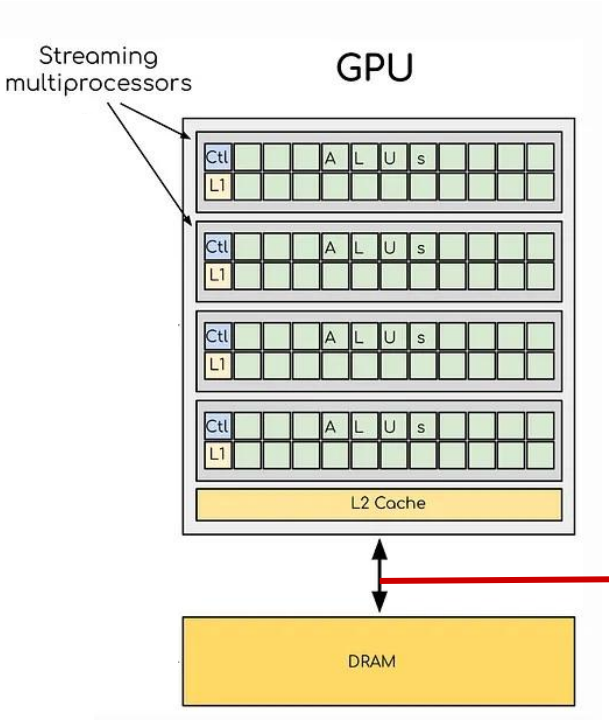
Expensive

Efficiency



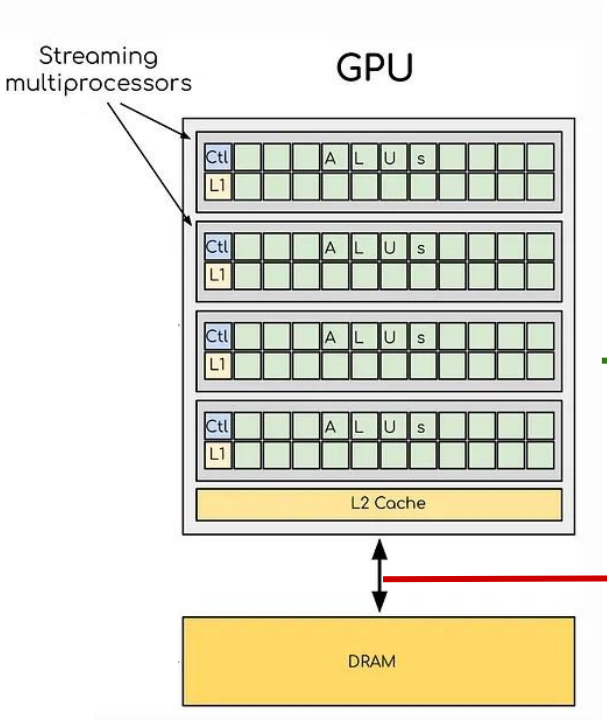
Expensive

Efficiency

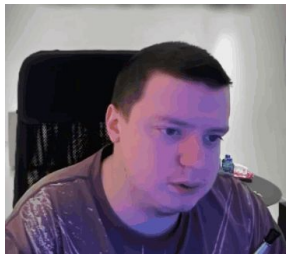


Expensive → use as little as possible

Efficiency

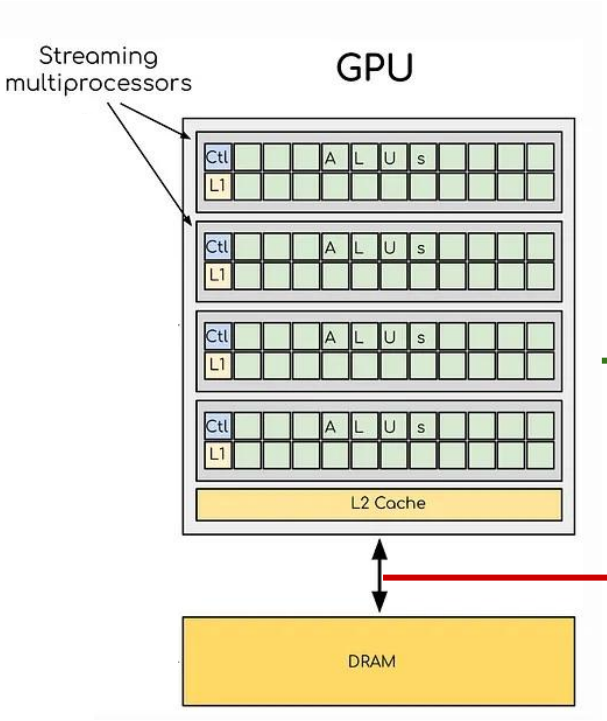


Cheap



Expensive → use as little as possible

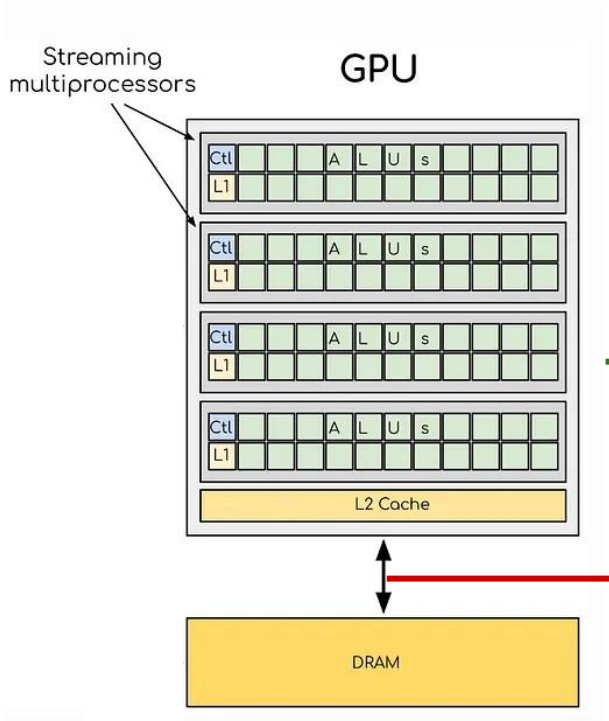
Efficiency



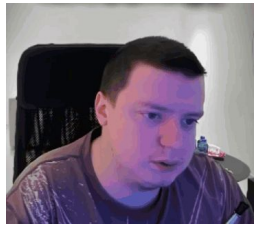
Cheap → use a little bit more of it

Expensive → use as little as possible

Efficiency

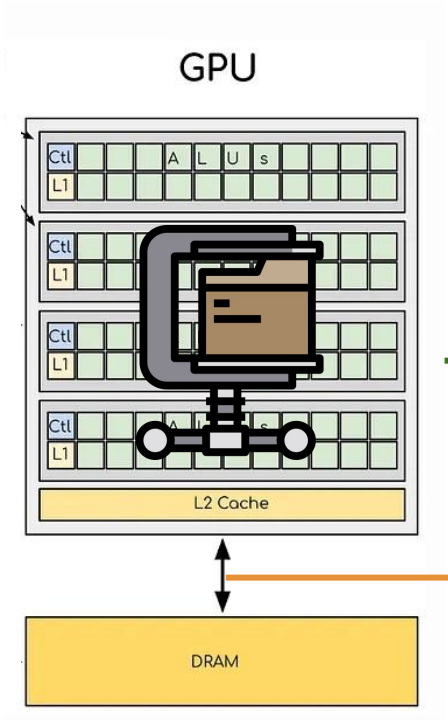


Cheap

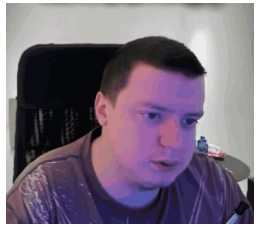


Expensive

Efficiency



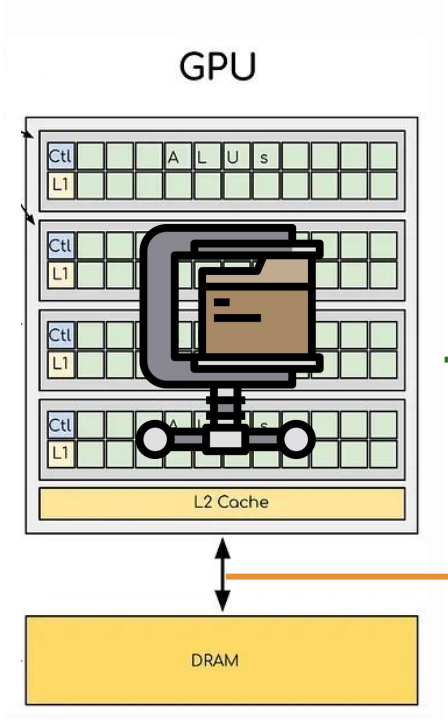
Cheap



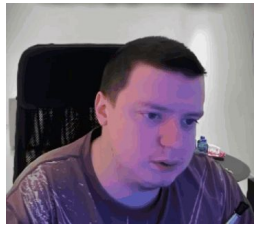
Affordable

Efficiency

- Move compressed data
- 👉 time/mem overhead
- 👉 performance drop



Cheap



Affordable

Compute and memory cost

How to measure “expensiveness”?

Time and energy required to complete the op

Compression in NNs

Compression in NNs

What is “data” in NNs? → weights (and activations)

Compression in NNs

What is the cost of data? → # values × # bits per value

Compression in NNs

What is the cost of data? → # values × # bits per value

E.g. a 1024×1024 image

Compression in NNs

What is the cost of data? → # values × # bits per value

E.g. a 1024×1024 image → 1024×1024 values × 32 bits per value

Compression in NNs

What is the cost of data? → # values × # bits per value

So, what are the levers we can work on to achieve compression?

Compression in NNs

What is the cost of data? → # values × # bits per value

So, what are the levers we can work on to achieve compression?

1) # values
2) # bits per value

Historical ages of compression in NNs

Compression → two levers

1) # values → pruning era (up to 2018)

Historical ages of compression in NNs

Compression → two levers

1) # values → pruning era (up to 2018)
2) # bits per value → quantization era (2018 onward)

Compression and compute

Compression → two levers (# values and # bits per value)

Ok, but how?

Compression and compute

Compression → two levers (# values and # bits per value)

Ok, but how?

Compression and compute

Compression → two levers (# values and # bits per value)

Ok, but how?

Compression and compute

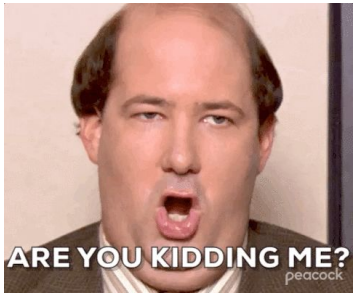
Compression → two levers (# values and # bits per value)

Ok, but how?

Compression and compute

Compression → two levers (# values and # bits per value)

Ok, but how?



Pruning

Pruning

Pruning

Any idea what pruning could mean in a deep neural network?

Pruning

Any idea what pruning could mean in a deep neural network?

1) Set a threshold

Pruning

Any idea what pruning could mean in a deep neural network?

1) Set a threshold
2) Zero-out all values ≤ threshold

Pruning

values vs. # bits per value

What lever is pruning using? Why?

Pruning

values vs. # bits per value

What lever is pruning using? Why?

Reducing the # of values!

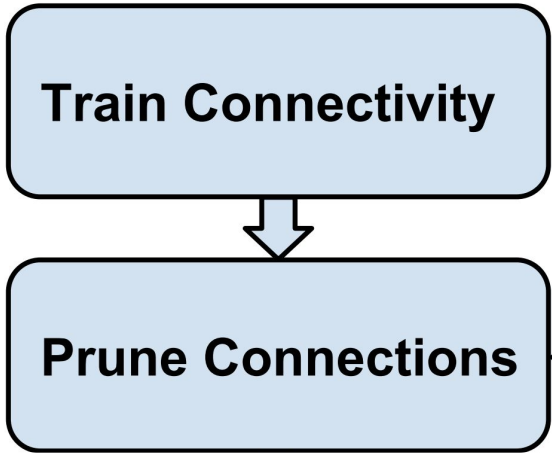
Han et al., NeurIPS 2015

Train Connectivity

1) Train CNN(s) from scratch

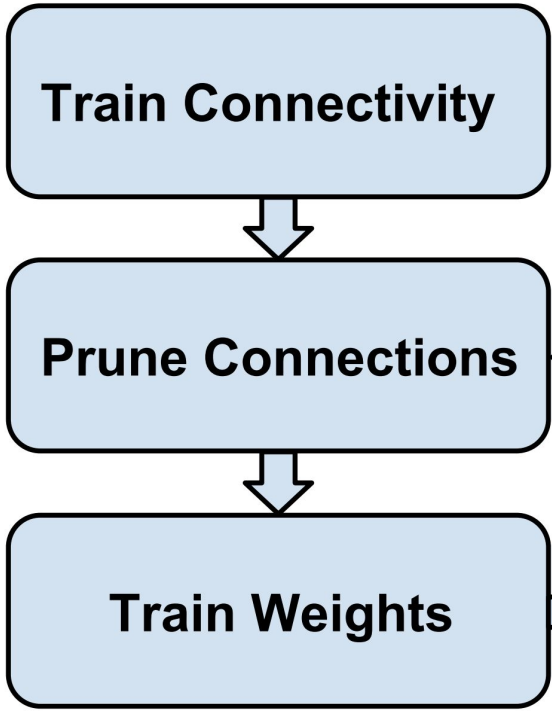
Han et al., NeurIPS 2015

1) Train CNN(s) from scratch
2) Zero-out weights below a defined threshold



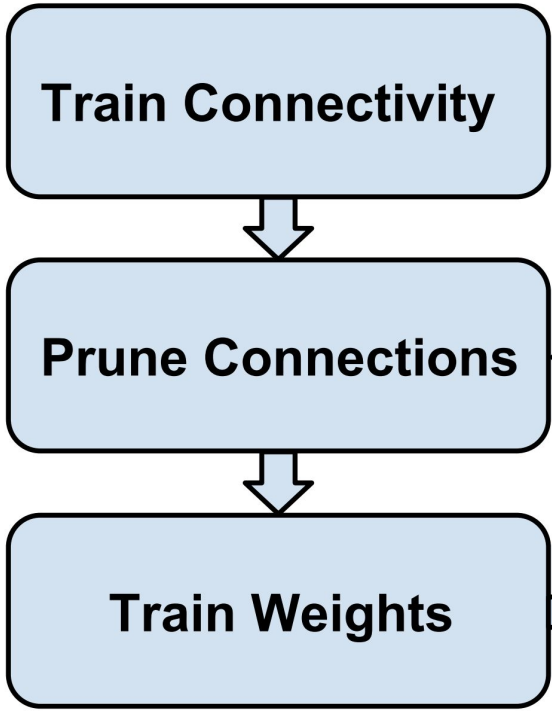
Han et al., NeurIPS 2015

1) Train CNN(s) from scratch
2) Zero-out weights below a defined threshold
3) Freeze those zero-out weights



Han et al., NeurIPS 2015

1) Train CNN(s) from scratch
2) Zero-out weights below a defined threshold
3) Freeze those zero-out weights
4) Continue training

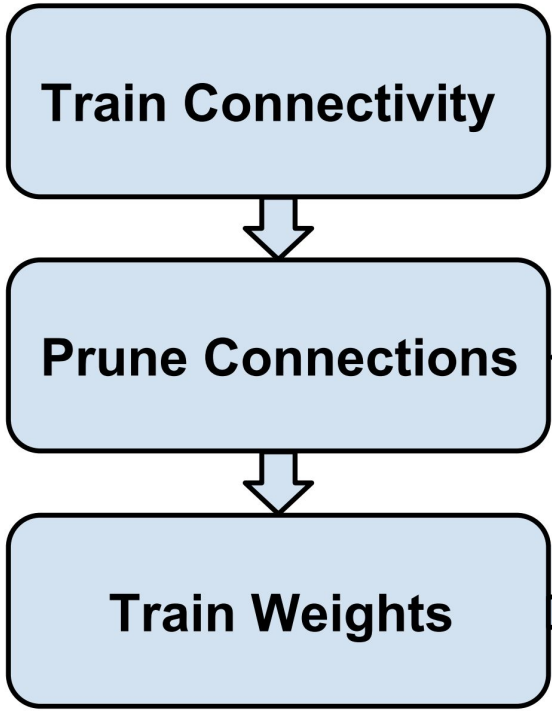


Han et al., NeurIPS 2015

“The final step retrains the network [...].

Pruned network w/o retraining → accuracy drop”

What do you think is happening? Why?

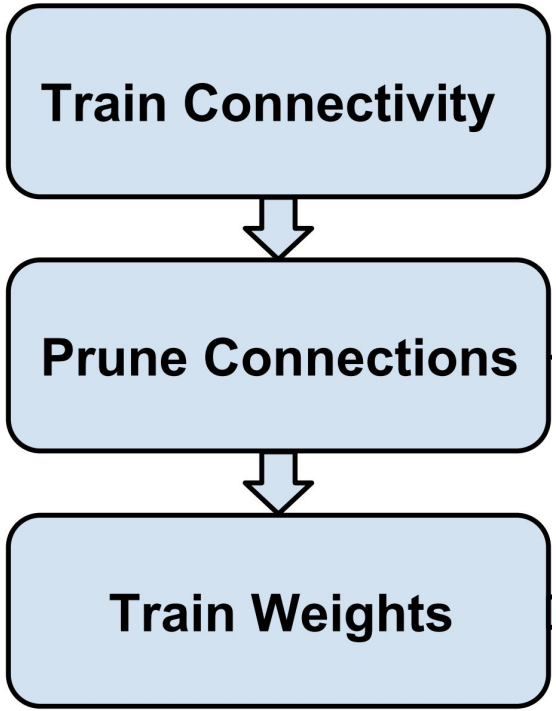


Han et al., NeurIPS 2015

“The final step retrains the network [...].

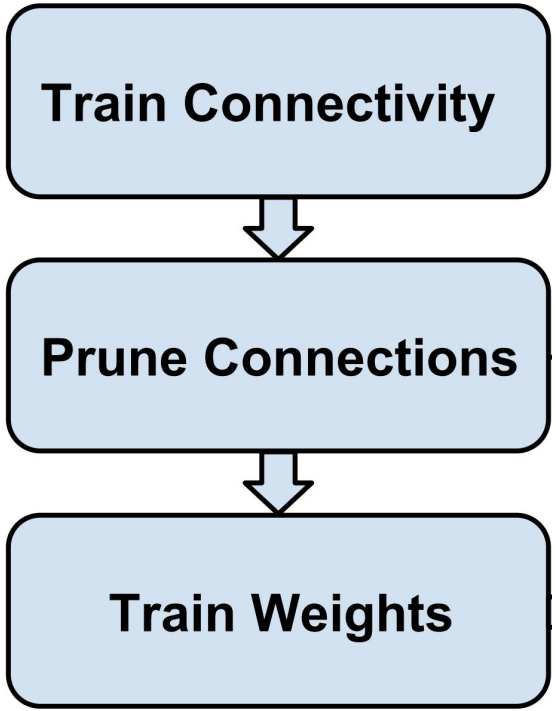
Pruned network w/o retraining → accuracy drop”

Retraining → network co-adapts to induced chages



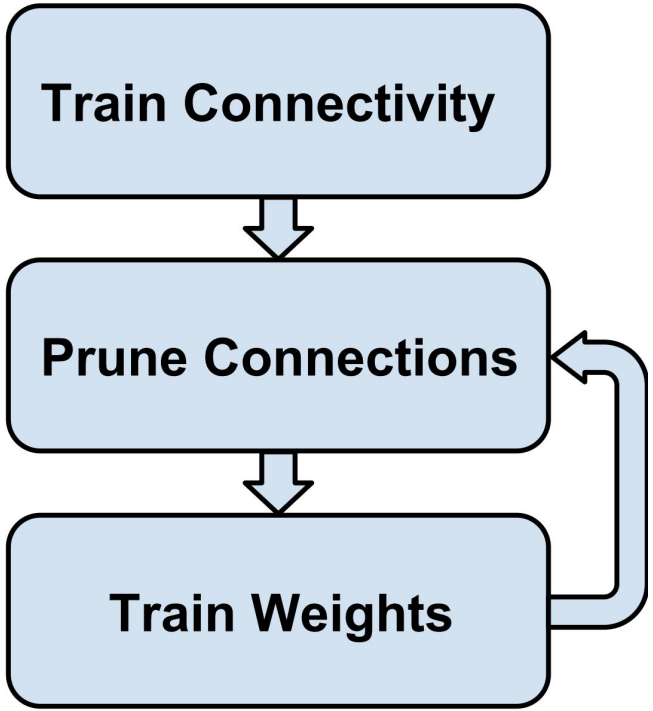
Han et al., NeurIPS 2015

1) Train CNN(s) from scratch
2) Zero-out weights below a defined threshold
3) Freeze those zero-out weights
4) Continue training

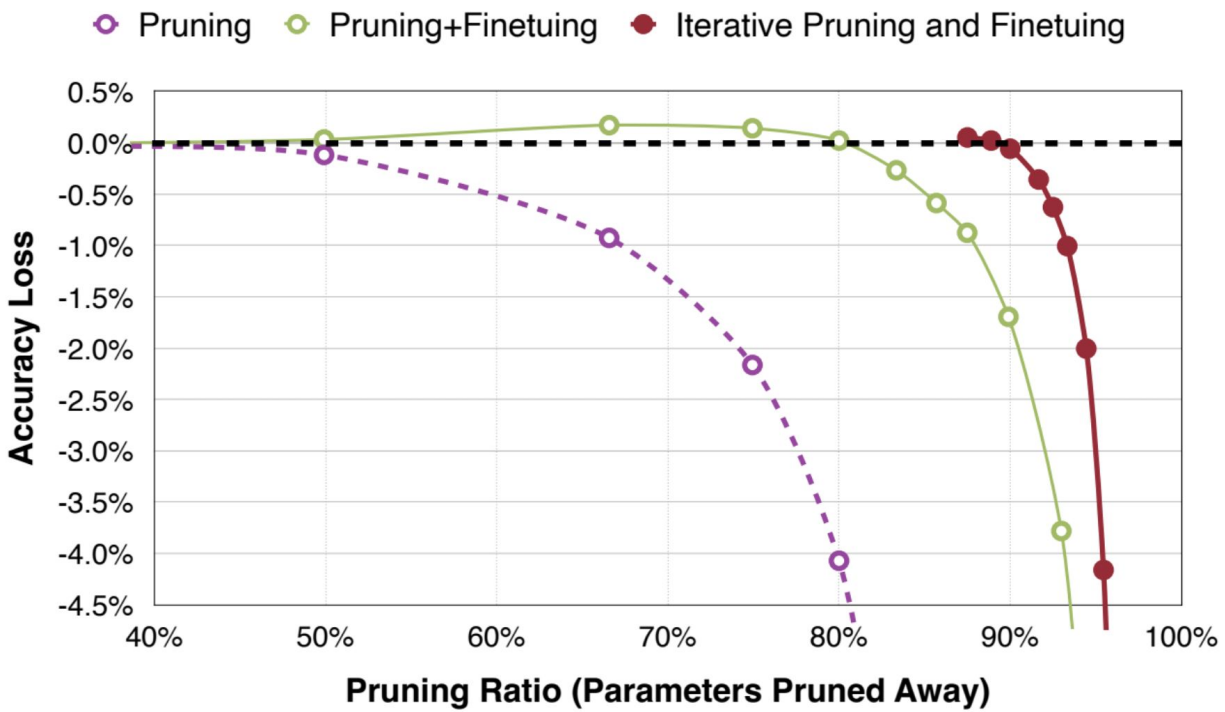


Han et al., NeurIPS 2015

1) Train CNN(s) from scratch
2) Zero-out weights below a defined threshold
3) Freeze those zero-out weights
4) Continue training
5) Go back to step 3)



Han et al., NeurIPS 2015

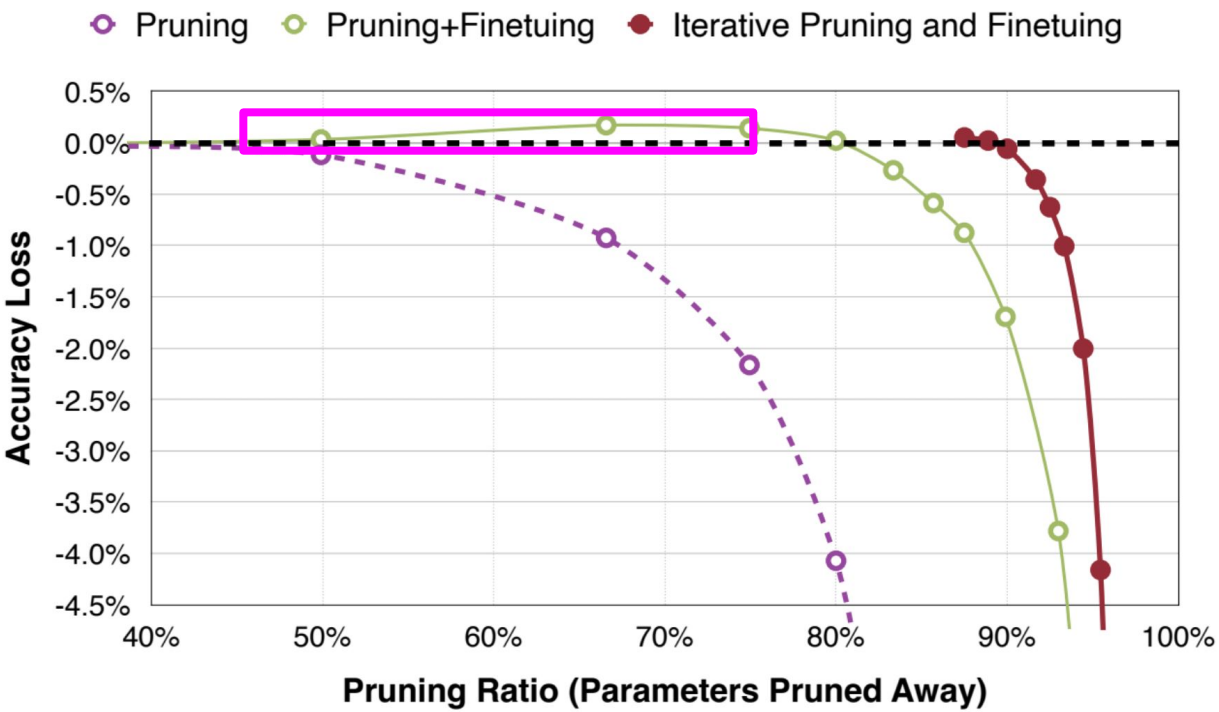


Han et al., NeurIPS 2015

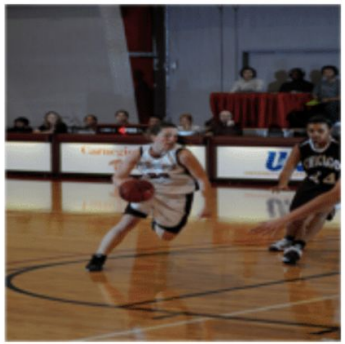
What do you notice?

Sometimes…

compression denoises NN!



Han et al., NeurIPS 2015



Baseline: a basketball player in a white uniform is playing with a ball .

Pruned 90%: a basketball player in a white uniform is playing with a basketball.

Baseline: a brown dog is running through a grassy field.

Pruned 90%: a brown dog is running through a grassy area.

Baseline: a man is riding a surfboard on a wave.

Pruned 90%: a man in a wetsuit is riding a wave on a beach.

Baseline: a soccer player in red is running in the field.

Pruned 95%: a man in a red shirt and black and white black shirt is running through a field.

Han et al., NeurIPS 2015

Neural Network	#Parameters			MACs
	Before Pruning	After Pruning	Reduction	Reduction
AlexNet	61 M	6.7 M	**9** ×	3 ×
VGG-16	138 M	10.3 M	**12** ×	5 ×
GoogleNet	7 M	2.0 M	**3.5** ×	5 ×
ResNet50	26 M	7.47 M	**3.4** ×	6.3 ×
SqueezeNet	1 M	0.38 M	**3.2** ×	3.5 ×

Pruning tip of the iceberg

Lotta stuff to decide. Unfortunately, out of scope for these lectures 😭

Pruning tip of the iceberg

Lotta stuff to decide. Unfortunately, out of scope for these lectures 😭

Criterion

Pruning tip of the iceberg

Lotta stuff to decide. Unfortunately, out of scope for these lectures 😭

Criterion → What weights to prune? How to select them?

Pruning tip of the iceberg

Lotta stuff to decide. Unfortunately, out of scope for these lectures 😭

- Granularity

Pruning tip of the iceberg

Lotta stuff to decide. Unfortunately, out of scope for these lectures 😭

Granularity → Pruning criterion might have a scope. Which scope to use?

Pruning tip of the iceberg

Lotta stuff to decide. Unfortunately, out of scope for these lectures 😭

Granularity → e.g. Criterion needs norm. Norm of what?

Pruning tip of the iceberg

Lotta stuff to decide. Unfortunately, out of scope for these lectures 😭

Granularity → e.g. Criterion needs norm. Norm of what?

Per-Tensor:

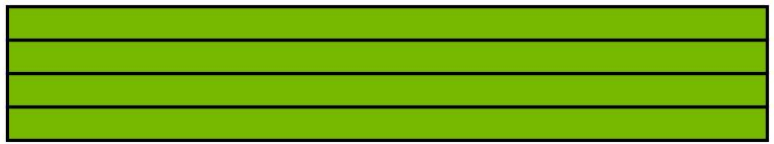
All elements

Pruning tip of the iceberg

Lotta stuff to decide. Unfortunately, out of scope for these lectures 😭

Granularity → e.g. Criterion needs norm. Norm of what?

Per-Channel:



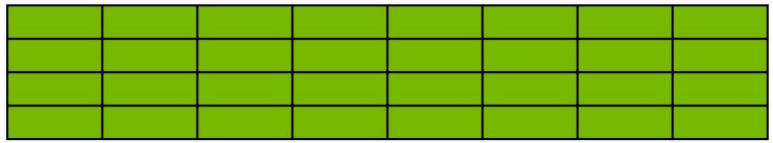
Each channel

Pruning tip of the iceberg

Lotta stuff to decide. Unfortunately, out of scope for these lectures 😭

Granularity → e.g. Criterion needs norm. Norm of what?

Per-Group:



Each group (small block)

Pruning tip of the iceberg

Lotta stuff to decide. Unfortunately, out of scope for these lectures 😭

Pruning ratio → How many weights to prune? How to select the ratio?

Pruning tip of the iceberg

Lotta stuff to decide. Unfortunately, out of scope for these lectures 😭

Pruning ratio → How many weights to prune? How to select the ratio?

Usually, heuristics or information-theory backed methods

Pruning tip of the iceberg

Lotta stuff to decide. Unfortunately, out of scope for these lectures 😭

Fine-tuning recipe → How to ft before/after pruning?

E.g. does sparsity promotion help with pruning? Does this loss work better?

Pruning tip of the iceberg

Lotta stuff we don’t have time to cover 😭

If interested, please refer to (video)lectures 3 and 4 of this 🐐 course 🚀

Quantization

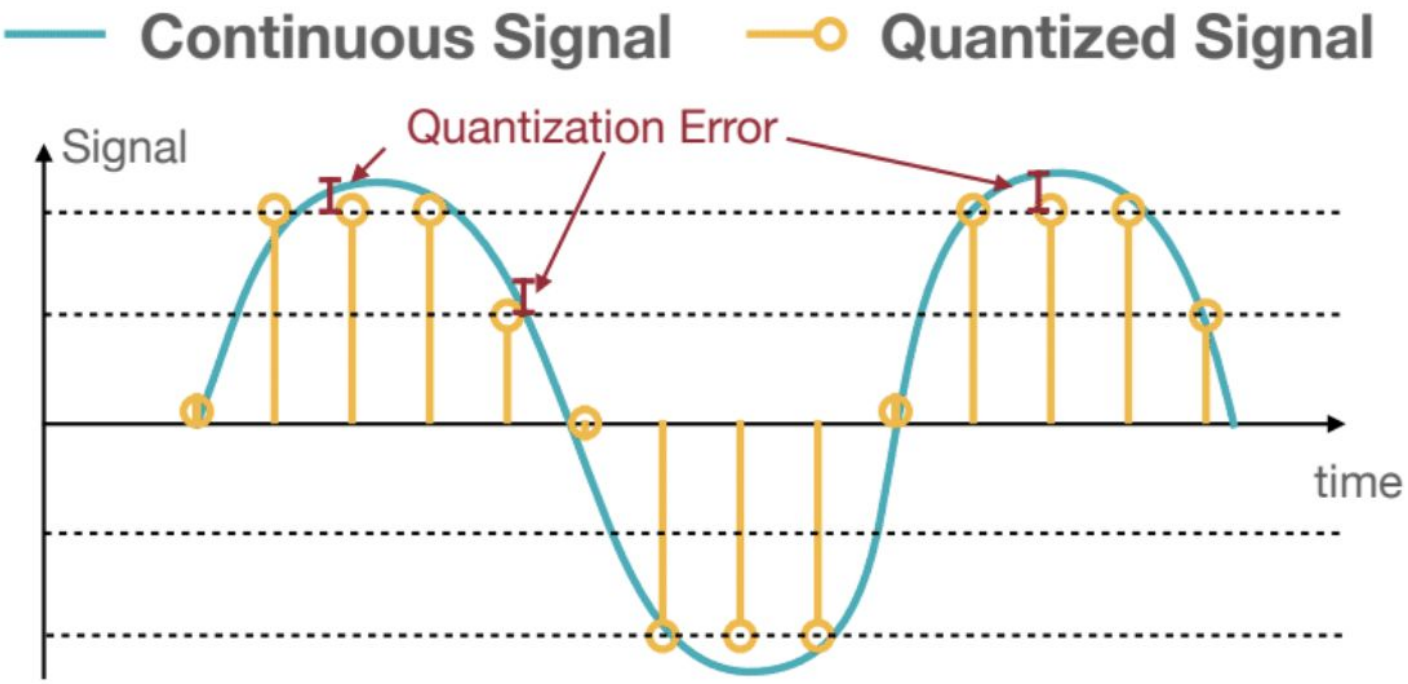
Definition

Continuous or large set of values (e.g. real numbers)

↓

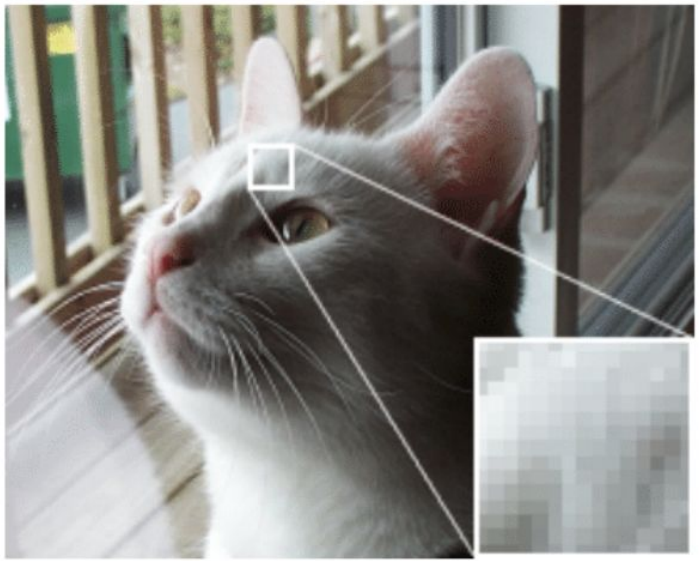
discrete set of values (e.g. integers)

Sinusoidal example

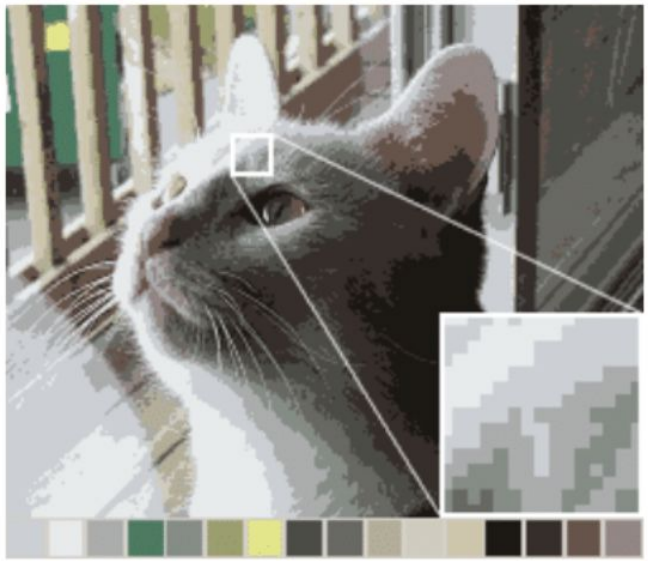


Color example

Original Image

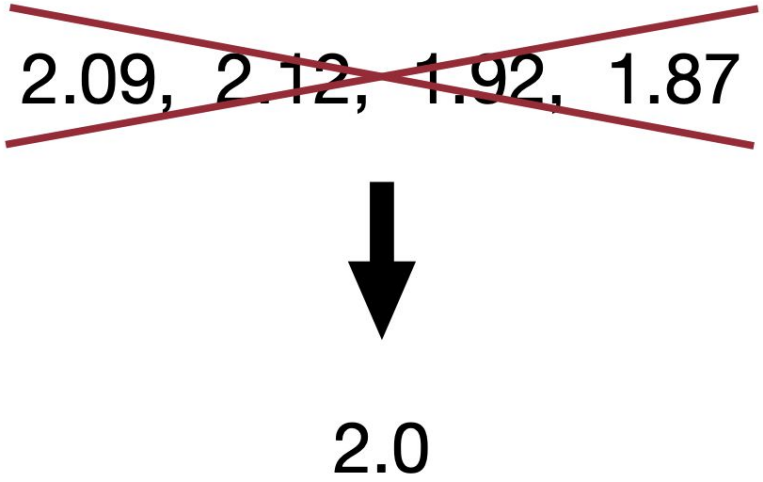


16-Color Image



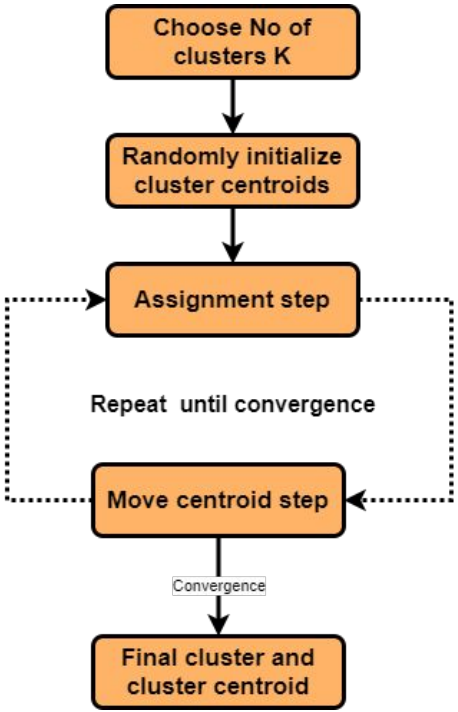
NN example

2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*

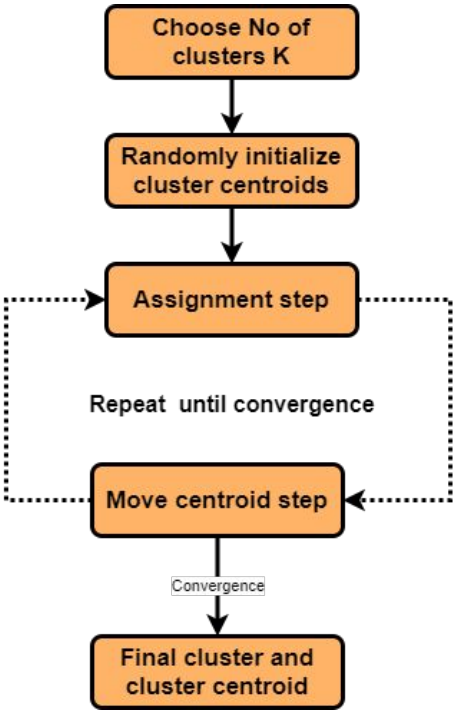


K-Means Quantization

Han et al., ICLR 2016: K-Means Brush-Up

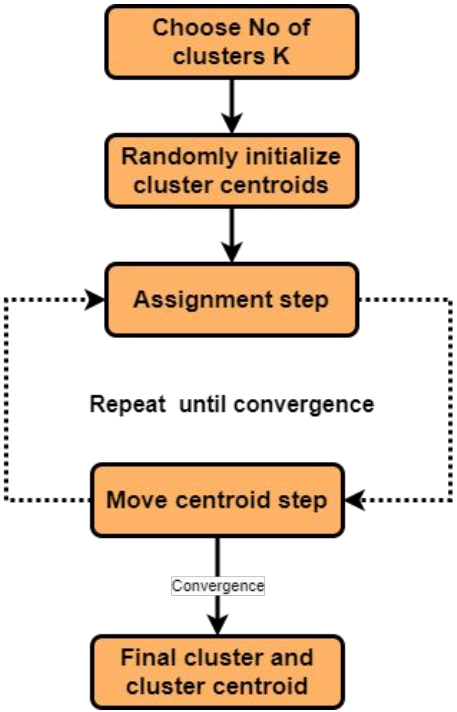


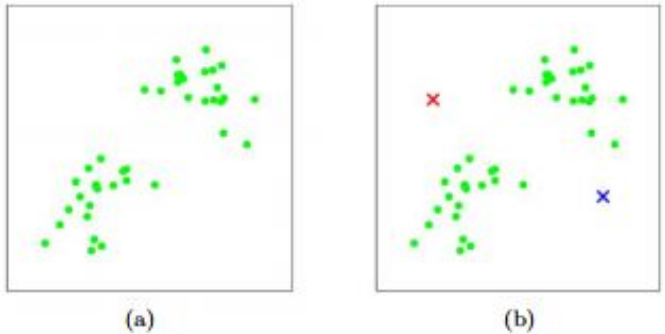
Han et al., ICLR 2016: K-Means Brush-Up



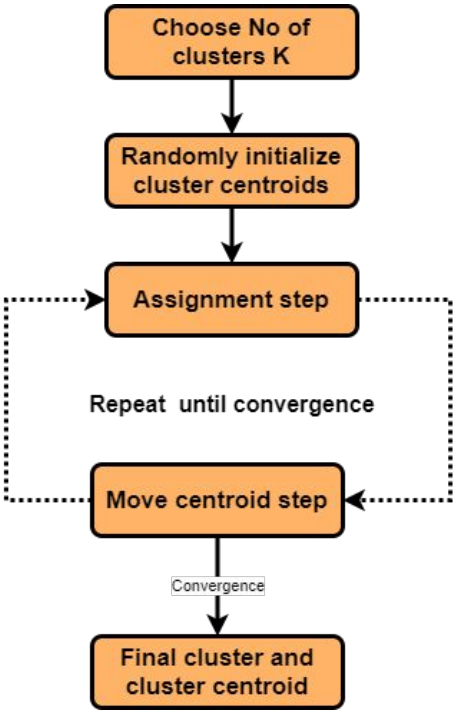
(a)

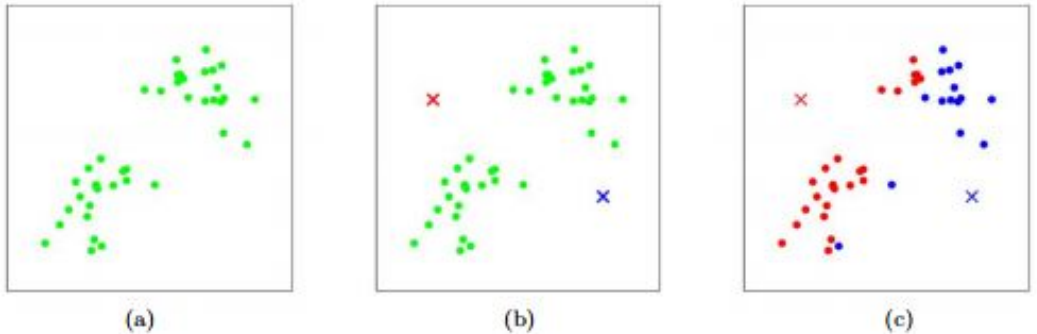
Han et al., ICLR 2016: K-Means Brush-Up



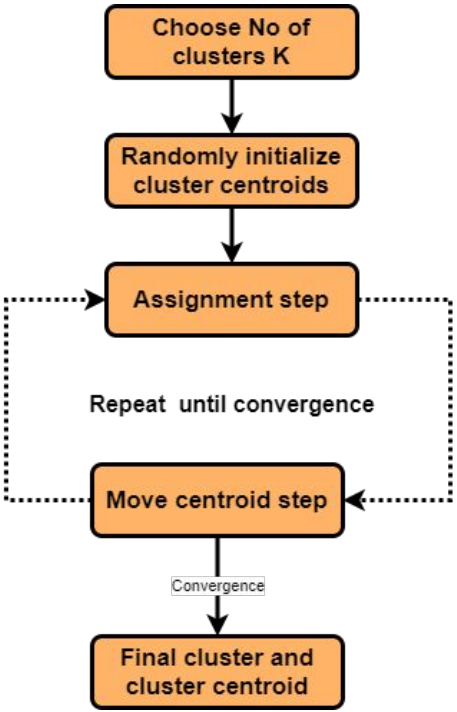


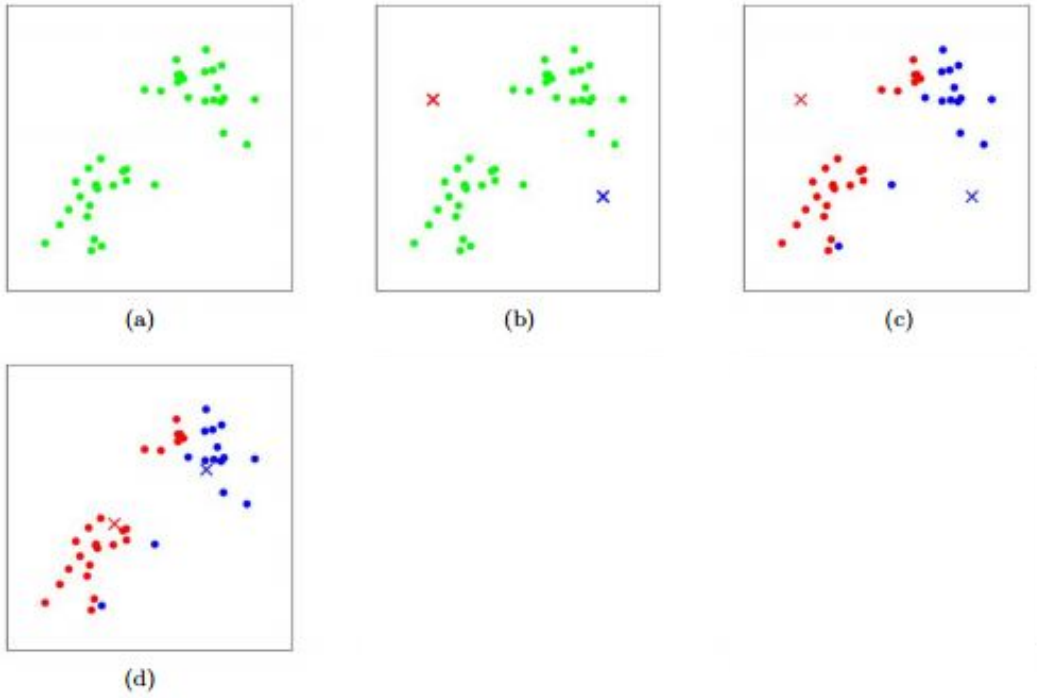
Han et al., ICLR 2016: K-Means Brush-Up



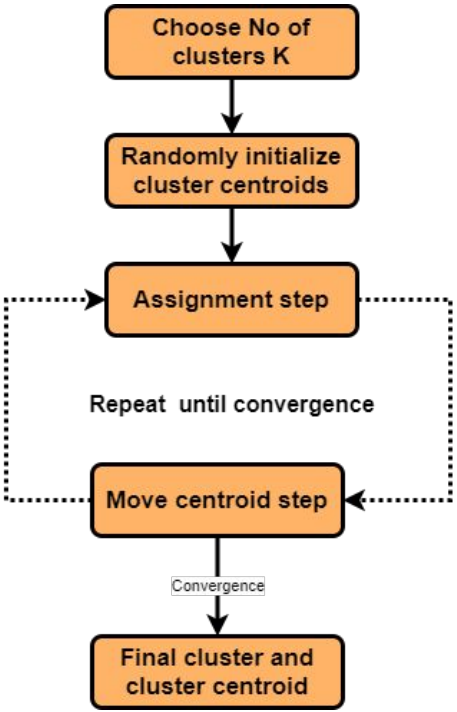


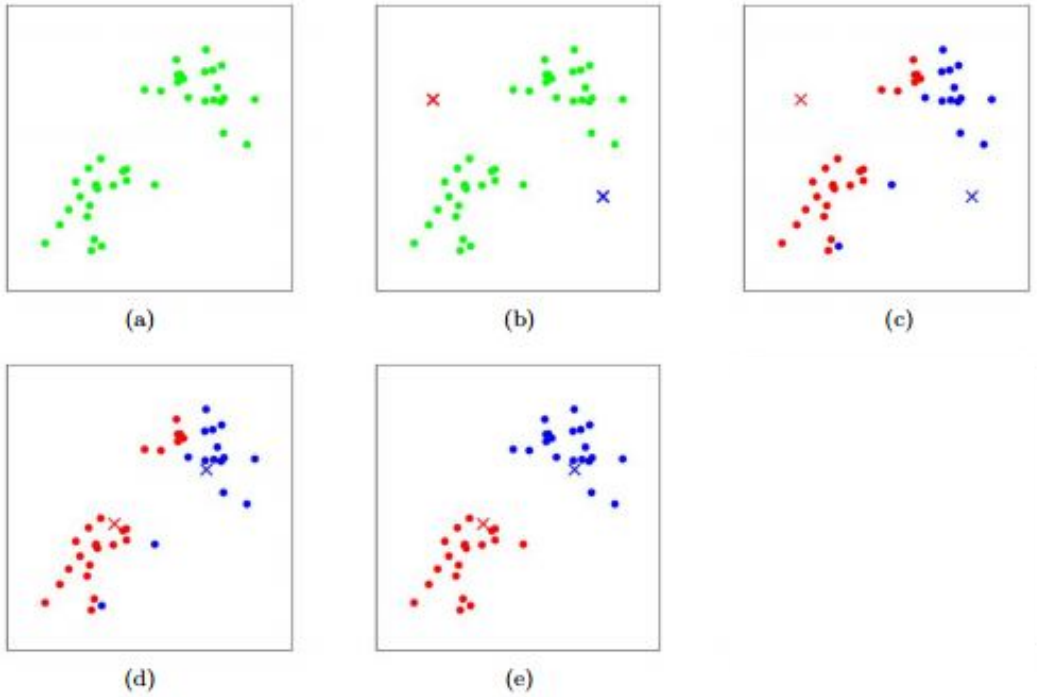
Han et al., ICLR 2016: K-Means Brush-Up



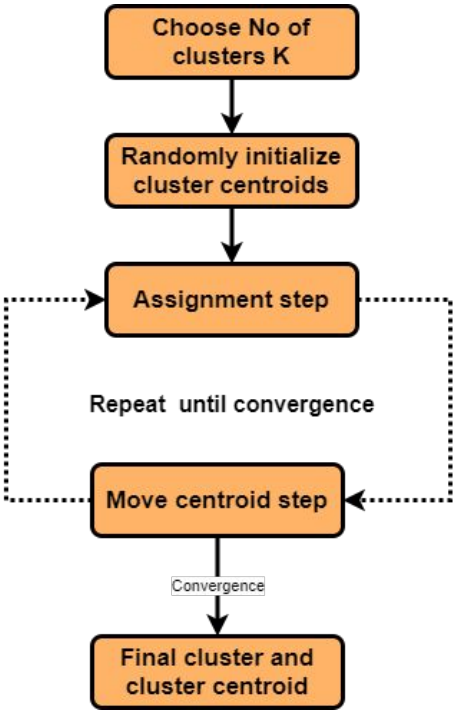


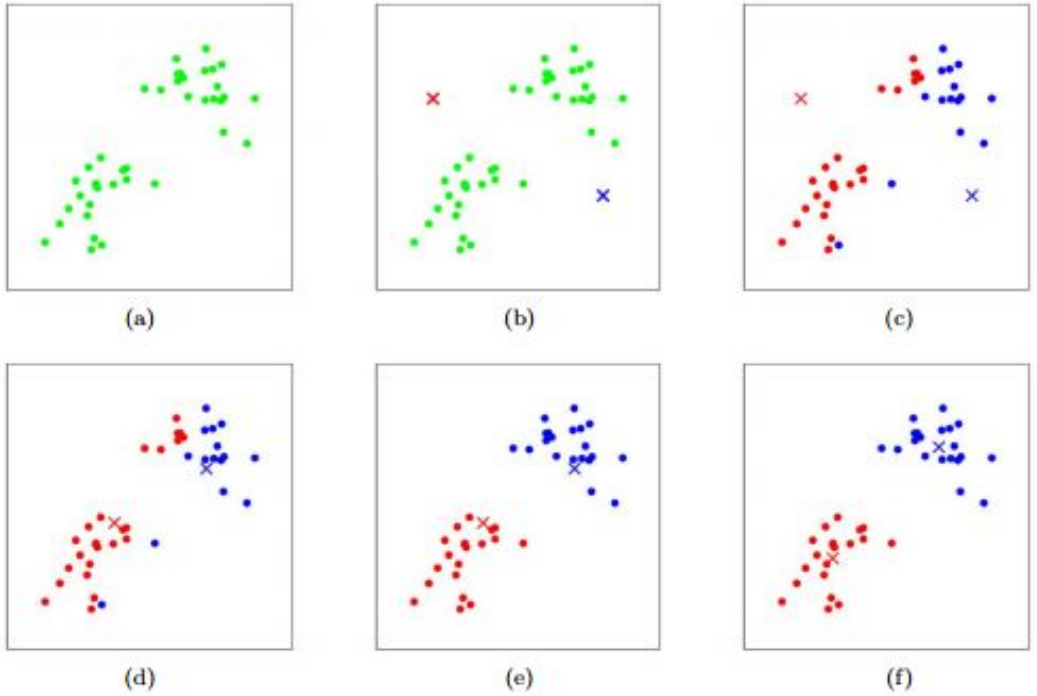
Han et al., ICLR 2016: K-Means Brush-Up





Han et al., ICLR 2016: K-Means Brush-Up





Han et al., ICLR 2016

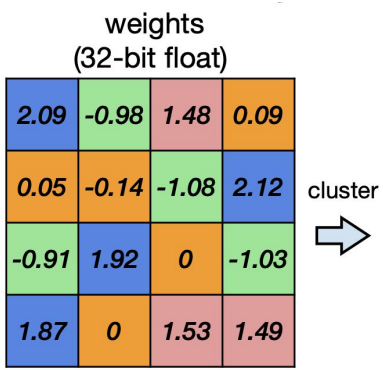
Quantize layer weights to centroids obtained by k-means clustering

Han et al., ICLR 2016

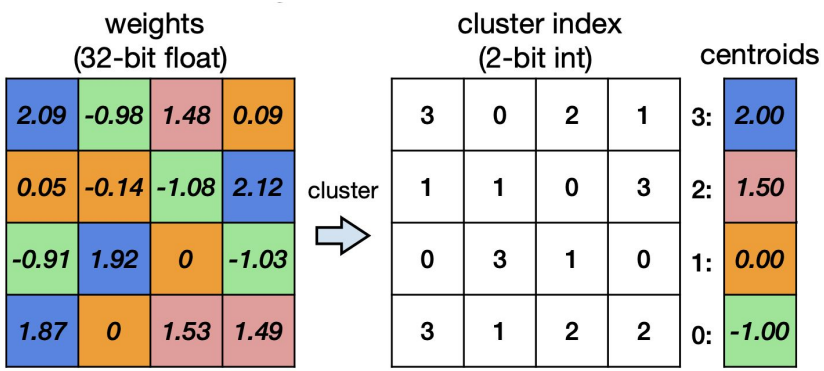
weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

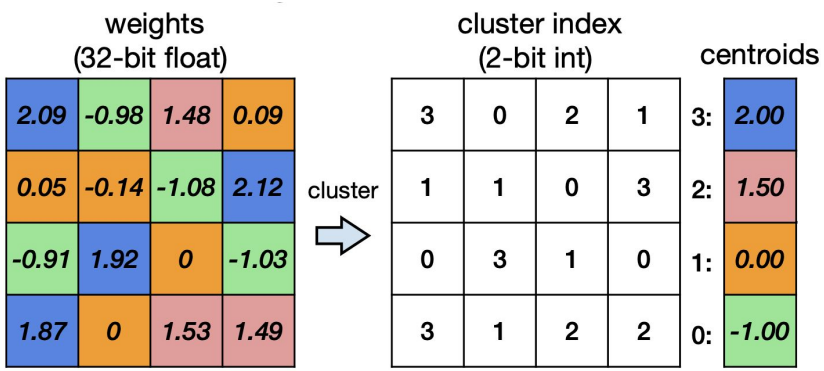
Han et al., ICLR 2016



Han et al., ICLR 2016

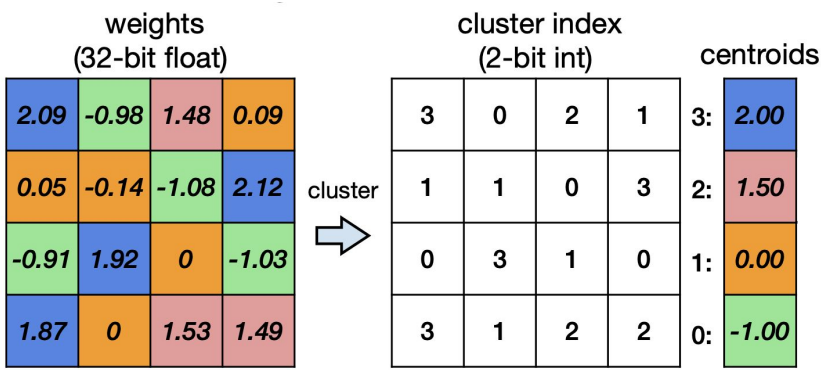


Han et al., ICLR 2016



Are we done?

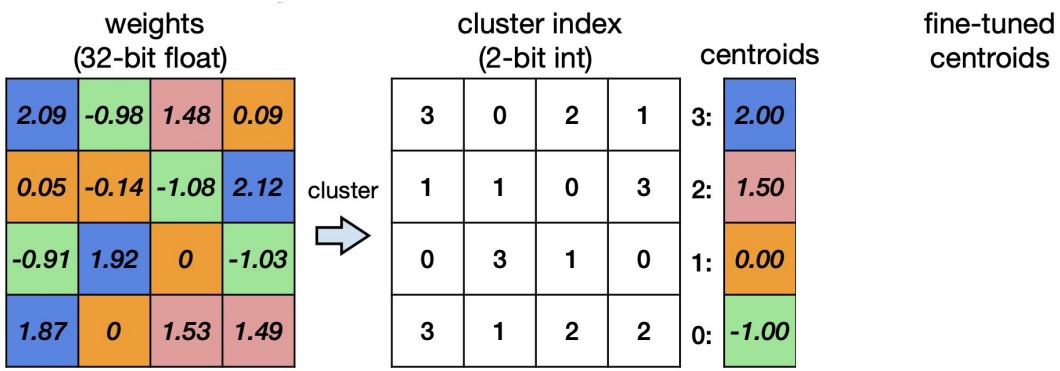
Han et al., ICLR 2016



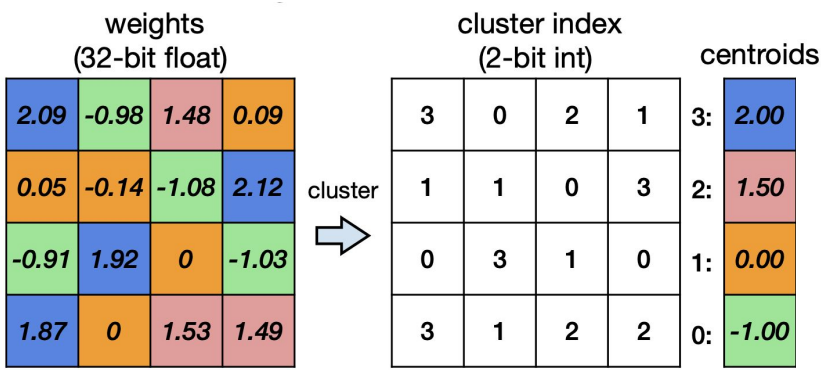
Are we done?

What do we need?

Han et al., ICLR 2016



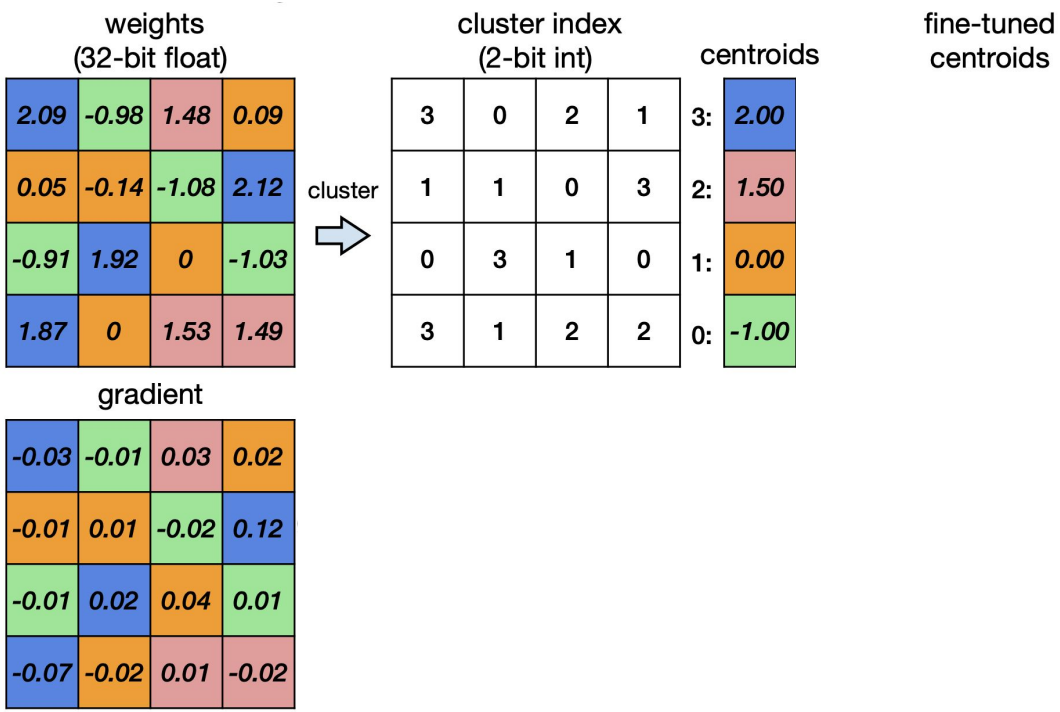
Han et al., ICLR 2016



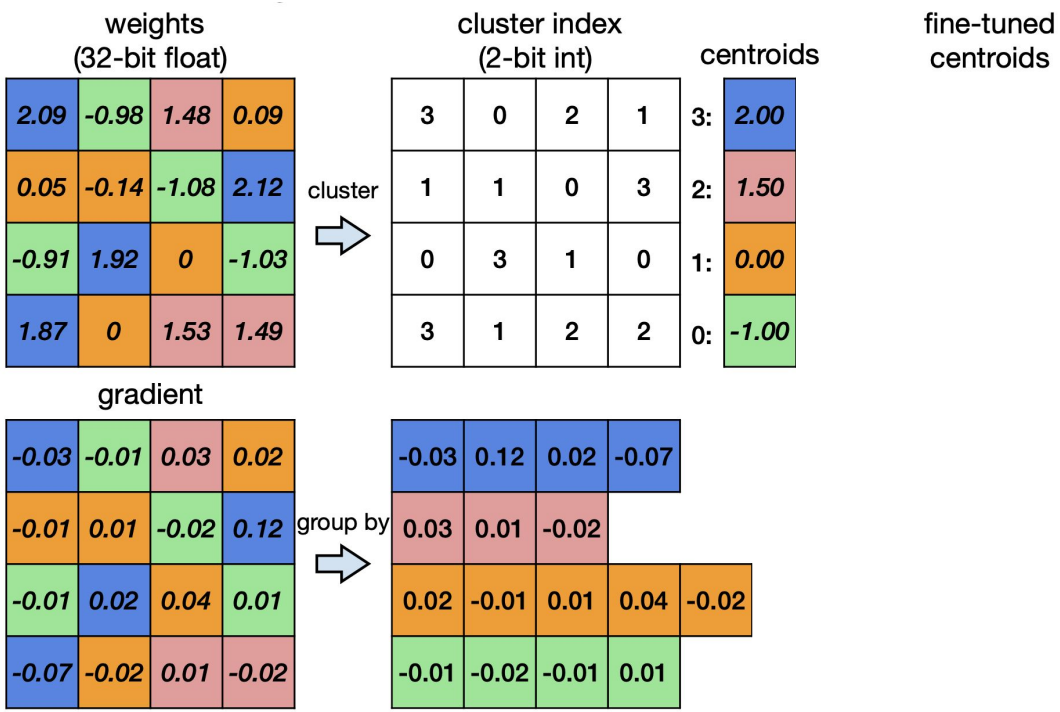
fine-tuned centroids

How to arrive to these?

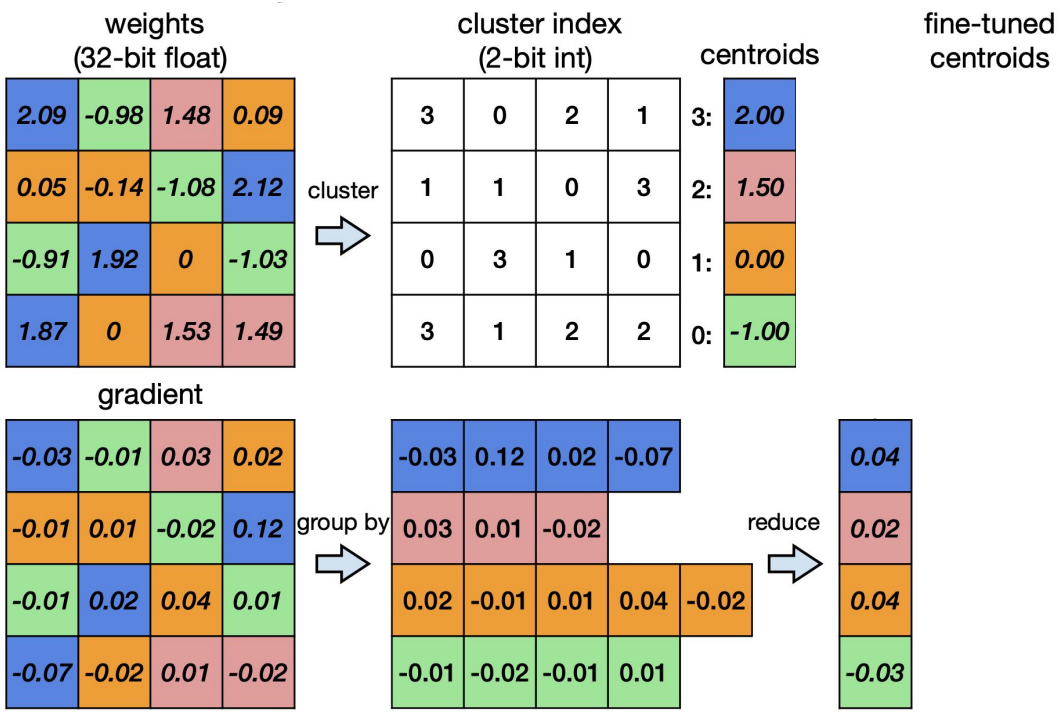
Han et al., ICLR 2016



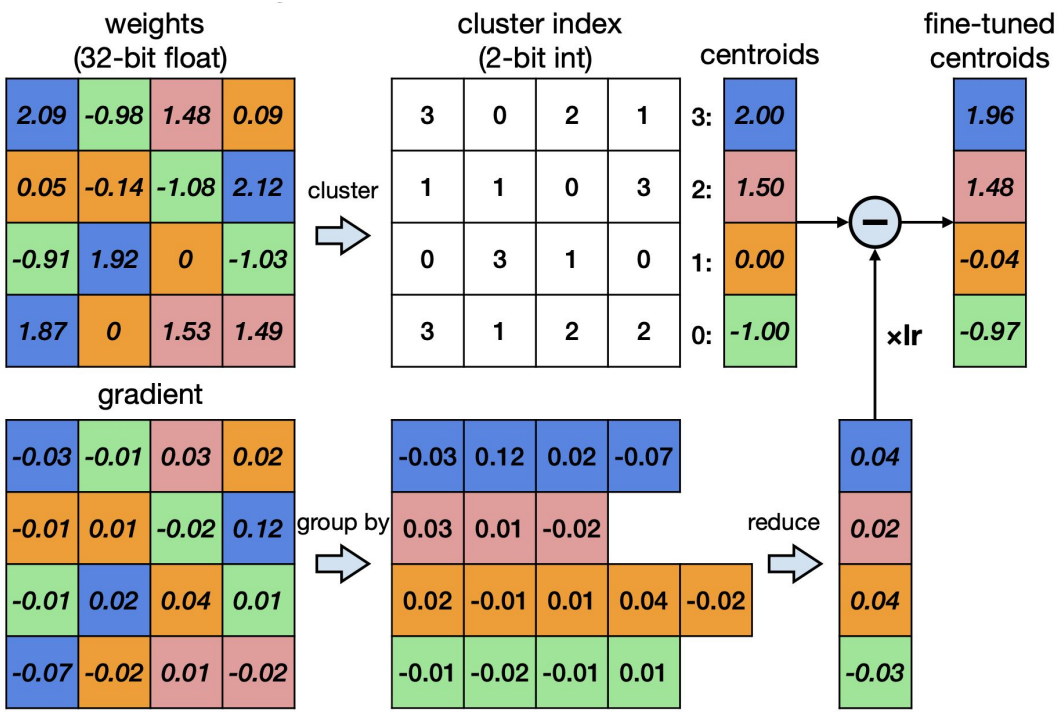
Han et al., ICLR 2016



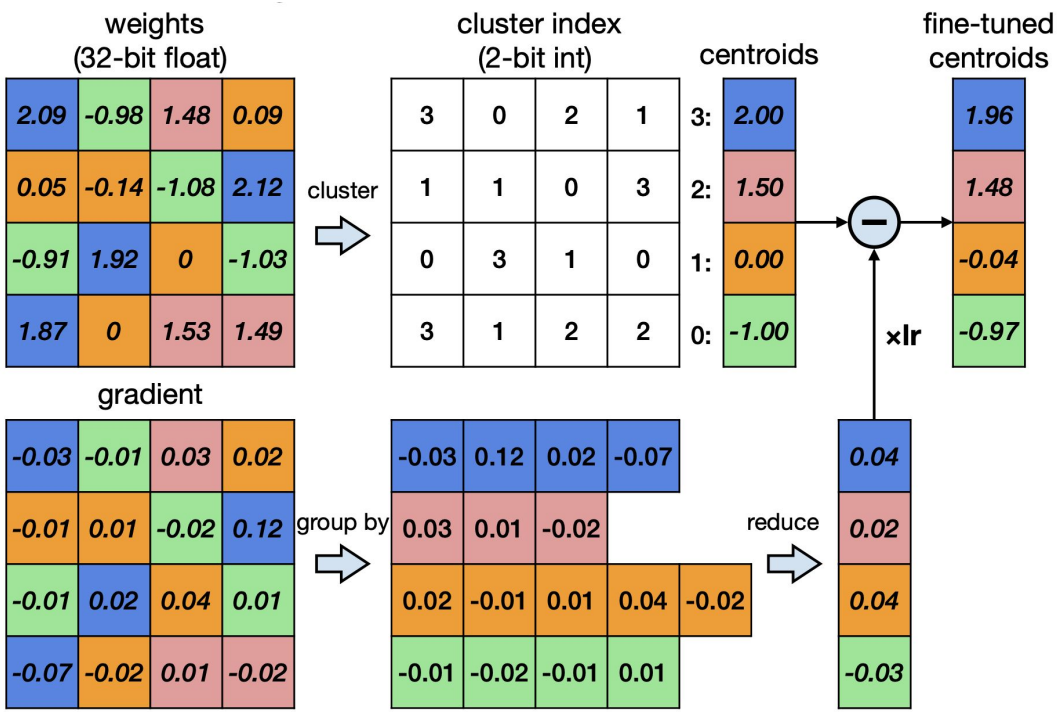
Han et al., ICLR 2016

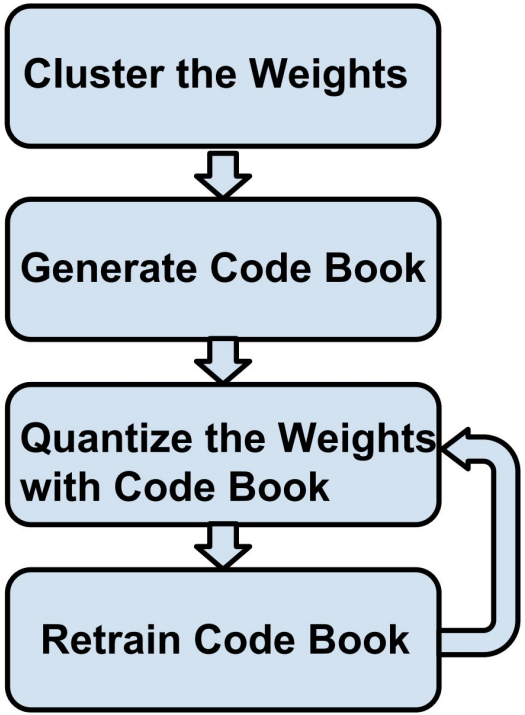


Han et al., ICLR 2016

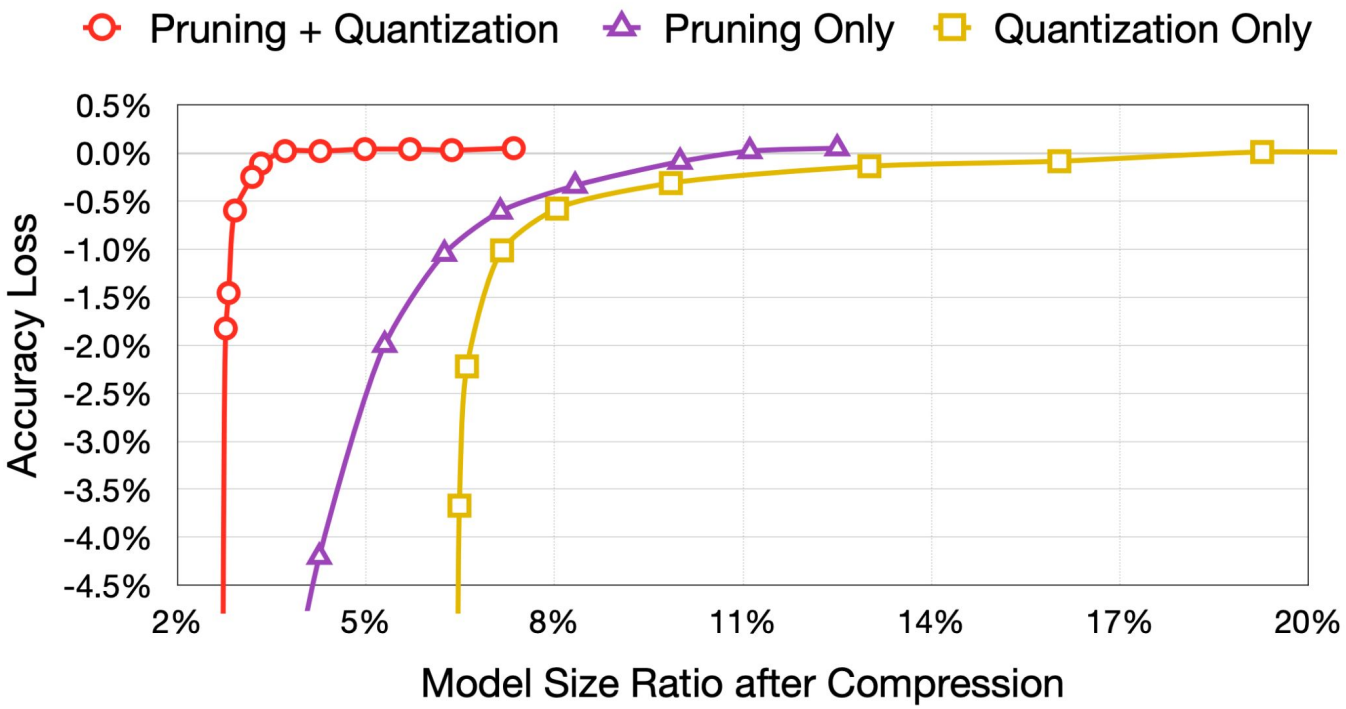


Han et al., ICLR 2016





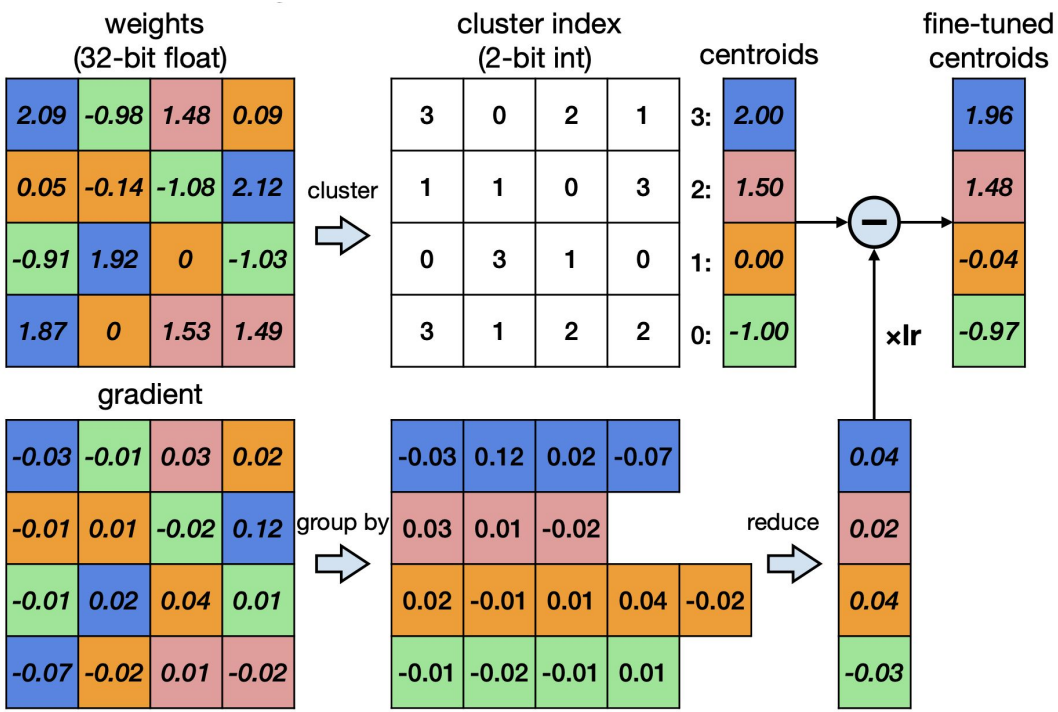
Han et al., ICLR 2016



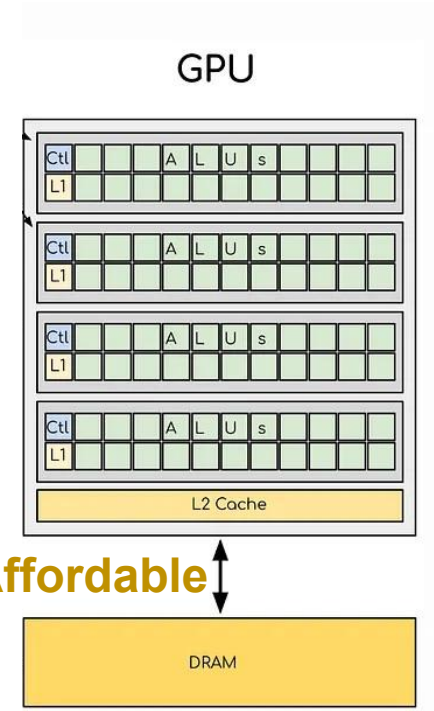
Han et al., ICLR 2016

Network	Original Size	Compressed Size	Compression Ratio	Original Accuracy	Compressed Accuracy
LeNet-300	1070KB	27KB	**40x**	98.36%	98.42%
LeNet-5	1720KB	44KB	**39x**	99.20%	99.26%
AlexNet	240MB	6.9MB	**35x**	80.27%	80.30%
VGGNet	550MB	11.3MB	**49x**	88.68%	89.09%
GoogleNet	28MB	2.8MB	**10x**	88.90%	88.92%
ResNet-18	44.6MB	4.0MB	**11x**	89.24%	89.28%

Quantization and compute

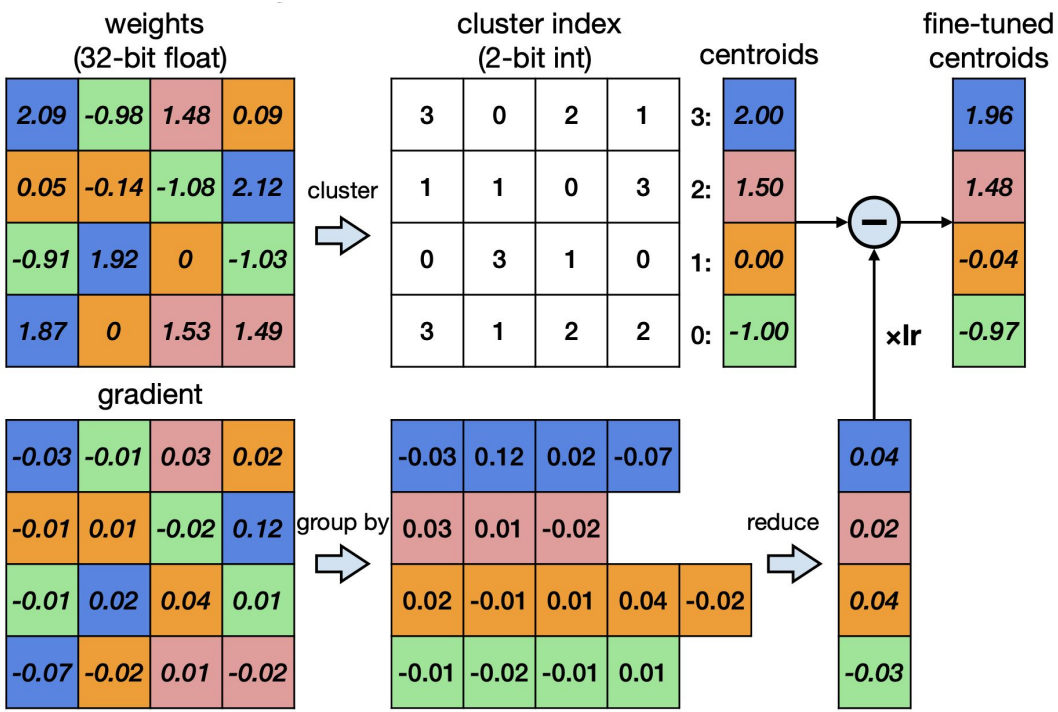


Cheap



Affordable

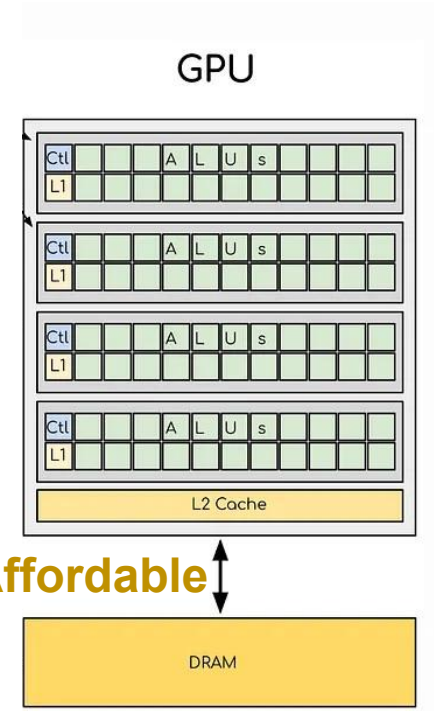
Quantization and compute



Cheap

↓

Cheaper?

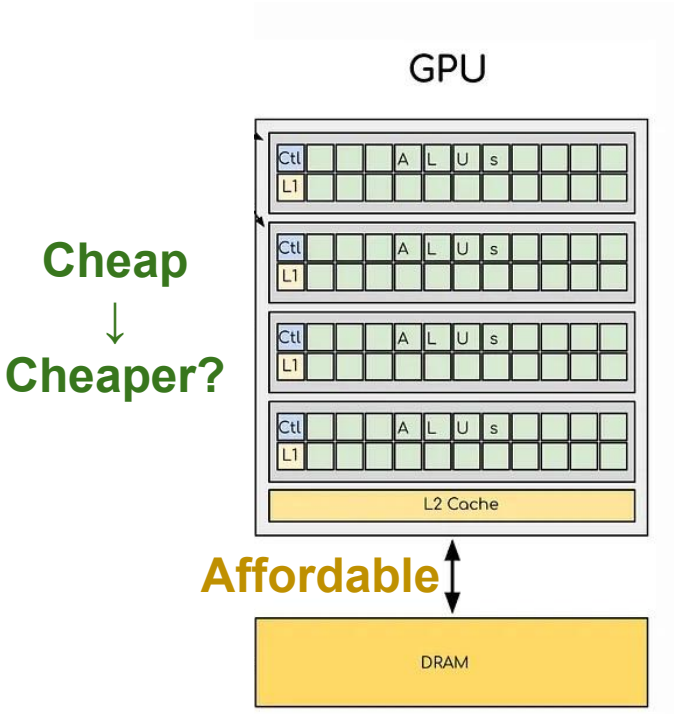


Quantization and compute

K-Means quantization → storage compression

Can we compress compute as well?

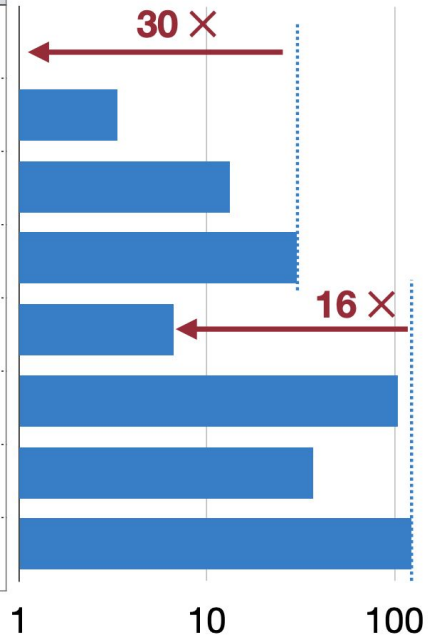
What does it mean to compress compute?



Cheap vs. cheaper

Operation	Energy [pJ]
8 bit int ADD	0.03
32 bit int ADD	0.1
16 bit float ADD	0.4
32 bit float ADD	0.9
8 bit int MULT	0.2
32 bit int MULT	3.1
16 bit float MULT	1.1
32 bit float MULT	3.7

Operations in 45nm 0.9V

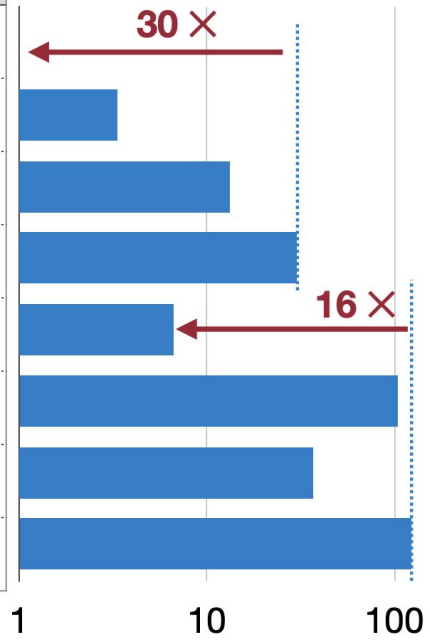


Cheap vs. cheaper

Lowering bits

Operation	Energy [pJ]
8 bit int ADD	0.03
32 bit int ADD	0.1
16 bit float ADD	0.4
32 bit float ADD	0.9
8 bit int MULT	0.2
32 bit int MULT	3.1
16 bit float MULT	1.1
32 bit float MULT	3.7

Operations in 45nm 0.9V



Cheap vs. cheaper

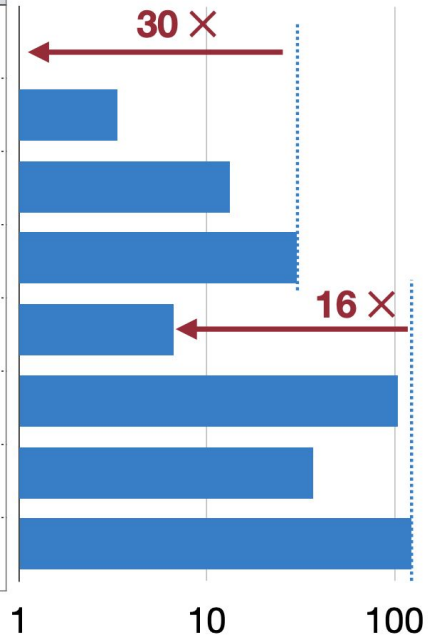
Lowering bits

+

Int dtype

Operation	Energy [pJ]
8 bit int ADD	0.03
32 bit int ADD	0.1
16 bit float ADD	0.4
32 bit float ADD	0.9
8 bit int MULT	0.2
32 bit int MULT	3.1
16 bit float MULT	1.1
32 bit float MULT	3.7

Operations in 45nm 0.9V



Cheap vs. cheaper

Lowering bits

+

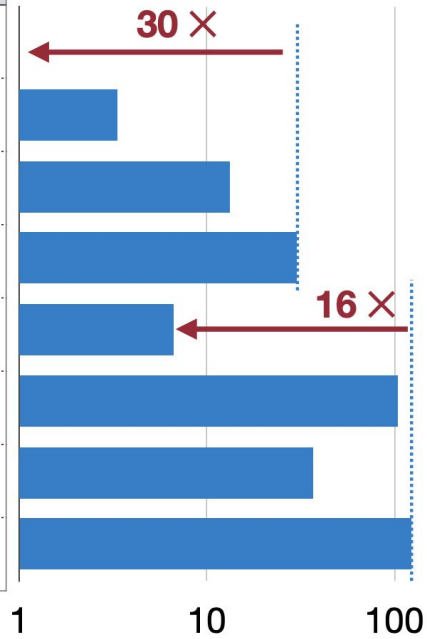
Int dtype

=

Cheaper ops!

Operation	Energy [pJ]
8 bit int ADD	0.03
32 bit int ADD	0.1
16 bit float ADD	0.4
32 bit float ADD	0.9
8 bit int MULT	0.2
32 bit int MULT	3.1
16 bit float MULT	1.1
32 bit float MULT	3.7

Operations in 45nm 0.9V



Cheap vs. cheaper

Lowering bits

+

Int dtype

=

Cheaper ops!

Can we use integer-only values?

Linear Affine Quantization

Jacob et al., CVPR 2018

weights
(32-bit float)

2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*

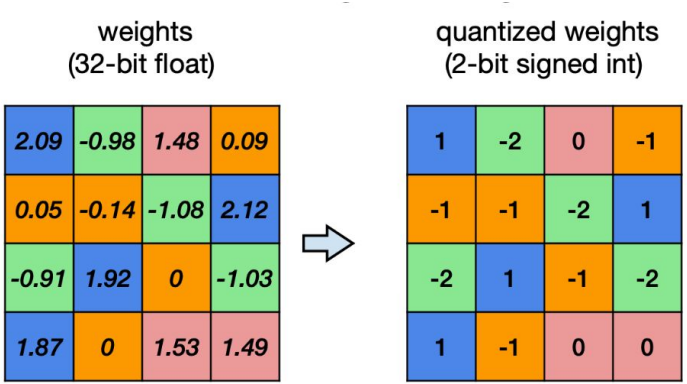
Jacob et al., CVPR 2018

weights
(32-bit float)

2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*

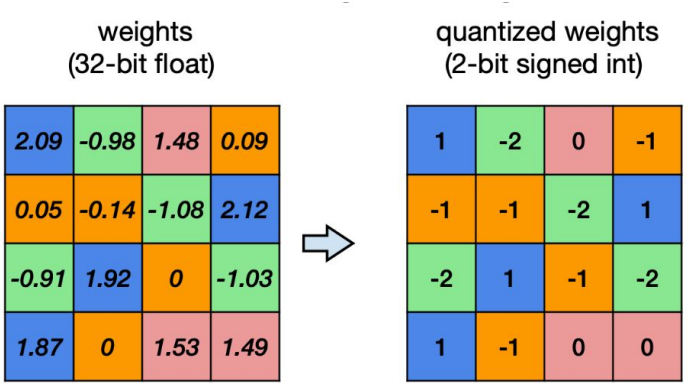
Binary	Decimal
01	1
00	0
11	-1
10	-2

Jacob et al., CVPR 2018



Binary	Decimal
01	1
00	0
11	-1
10	-2

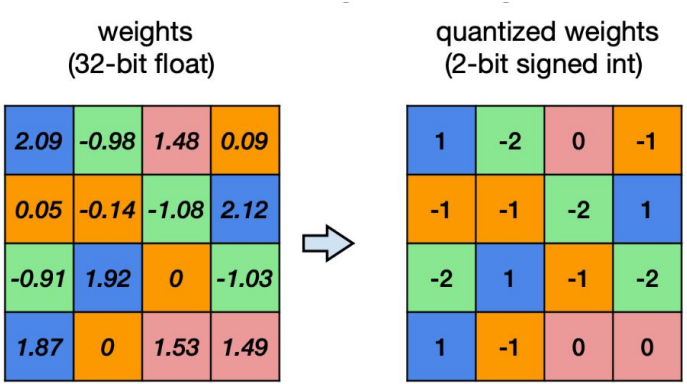
Jacob et al., CVPR 2018



How to quantize weights?

Binary	Decimal
01	1
00	0
11	-1
10	-2

Jacob et al., CVPR 2018

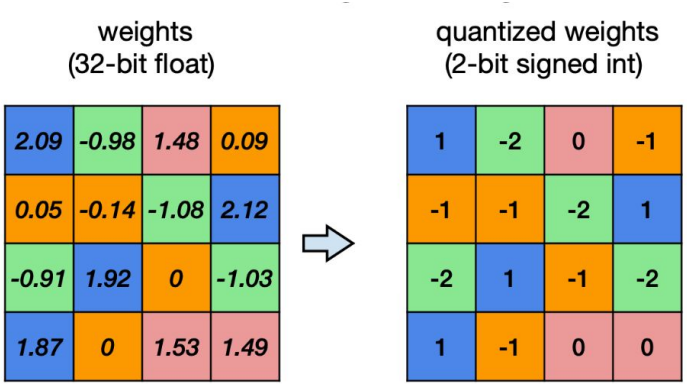


How to quantize weights?

$$q = \text{clip}\left(\text{round}\left(\frac{r}{S} + Z\right),\ q_{\min},\ q_{\max}\right)$$

Binary	Decimal
01	1
00	0
11	-1
10	-2

Jacob et al., CVPR 2018



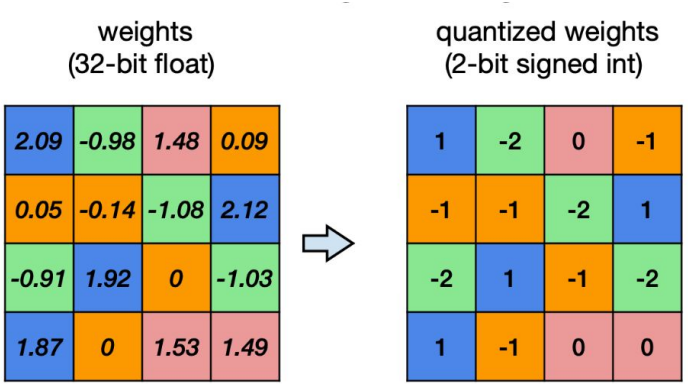
How to quantize weights?

$$q = \text{clip}\left(\text{round}\left(\frac{r}{S} + Z\right),\ q_{\min},\ q_{\max}\right)$$

r → float value ✅

Binary	Decimal
01	1
00	0
11	-1
10	-2

Jacob et al., CVPR 2018



How to quantize weights?

$$q = \text{clip}\left(\text{round}\left(\frac{r}{S} + Z\right),\ q_{\min},\ q_{\max}\right)$$

r → float value ✅

q → integer-quantized value 🪄

Binary	Decimal
01	1
00	0
11	-1
10	-2

Jacob et al., CVPR 2018

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

quantized weights
(2-bit signed int)

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

Binary	Decimal
01	1
00	0
11	-1
10	-2

How to quantize weights?

$$q = \text{clip}\left(\text{round}\left(\frac{r}{S} + Z\right),\ q_{\min},\ q_{\max}\right)$$

r → float value ✅

q → integer-quantized value 🪄

S → scale 🤷

Jacob et al., CVPR 2018

weights
(32-bit float)

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

quantized weights
(2-bit signed int)

1	-2	0	-1
-1	-1	-2	1
-2	1	-1	-2
1	-1	0	0

Binary	Decimal
01	1
00	0
11	-1
10	-2

How to quantize weights?

$$q = \text{clip}\left(\text{round}\left(\frac{r}{S} + Z\right),\ q_{\min},\ q_{\max}\right)$$

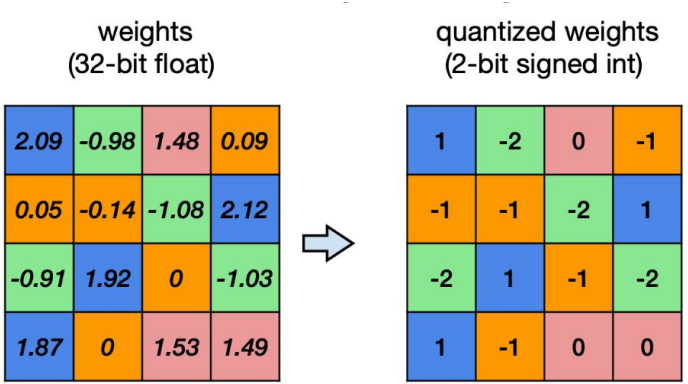
r → float value ✅

q → integer-quantized value 🪄

S → scale 🤷

Z → zero point 🤷

Jacob et al., CVPR 2018



Binary	Decimal
01	1
00	0
11	-1
10	-2

How to quantize weights?

$$q = \text{clip}\left(\text{round}\left(\frac{r}{S} + Z\right),\ q_{\min},\ q_{\max}\right)$$

r → float value ✅

q → integer-quantized value 🪄

S → scale 🤷

Z → zero point 🤷

$q_{\{min,max\}}$ → integer range ✅

Jacob et al., CVPR 2018

How to quantize weights?

$$q = \text{clip}\left(\text{round}\left(\frac{r}{S} + Z\right),\ q_{\min},\ q_{\max}\right)$$

r → float value ✅

q → integer-quantized value 🪄

S → scale 🤷

Z → zero point 🤷

$q_{\{min,max\}}$ → integer dtype range ✅

S → scale 🤷

$$S = \frac{r_{\max} - r_{\min}}{q_{\max} - q_{\min}}$$

$r_{\{min,max\}}$ → original float value range ✅

$$q_{\min} = -2, \quad q_{\max} = 1$$

$$r_{\min} = -1.08, \quad r_{\max} = 2.12$$

$$S = \frac{2.12 - (-1.08)}{1 - (-2)} = \frac{3.20}{3} \approx 1.07$$

Jacob et al., CVPR 2018

Jacob et al., CVPR 2018

Hella lot of wall text…

Let’s visualize stuff!

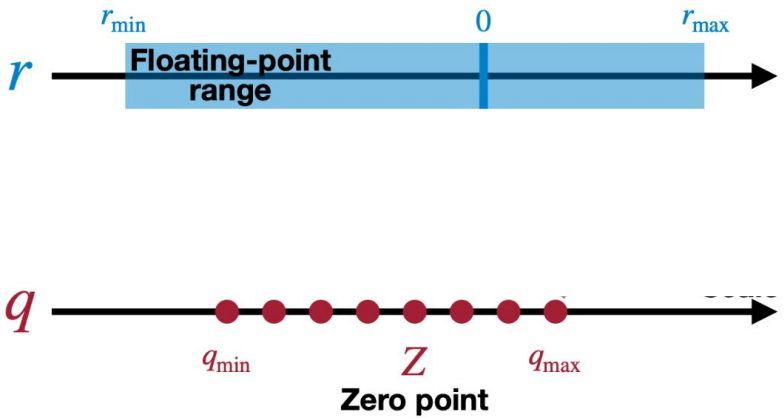
Jacob et al., CVPR 2018

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

Jacob et al., CVPR 2018

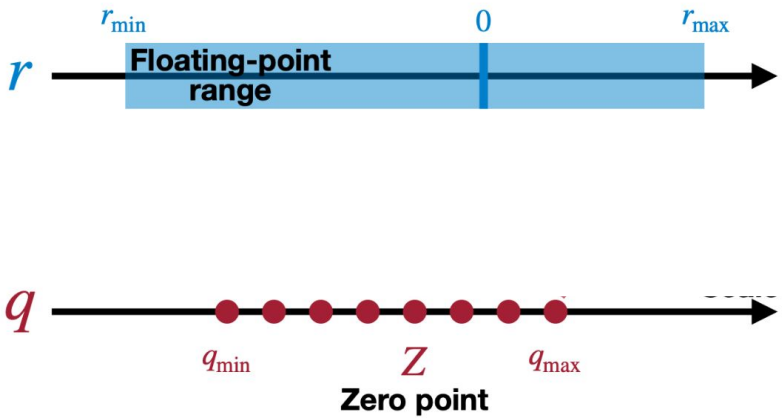
2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*

Jacob et al., CVPR 2018



2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

Jacob et al., CVPR 2018

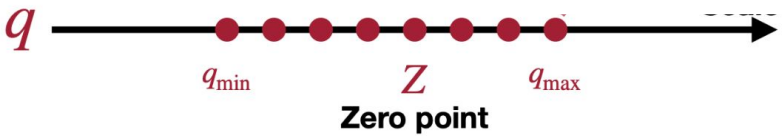


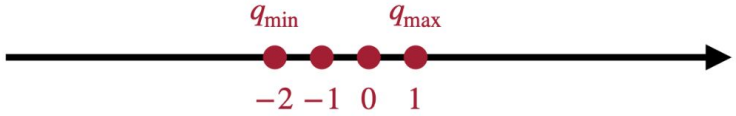
2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*

Binary	Decimal
01	1
00	0
11	-1
10	-2

Jacob et al., CVPR 2018

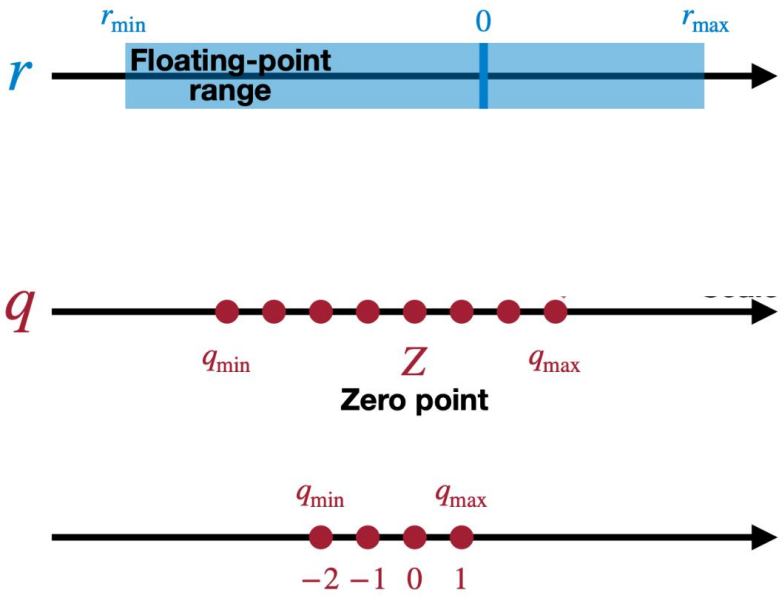
2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*





Binary	Decimal
01	1
00	0
11	-1
10	-2

Jacob et al., CVPR 2018

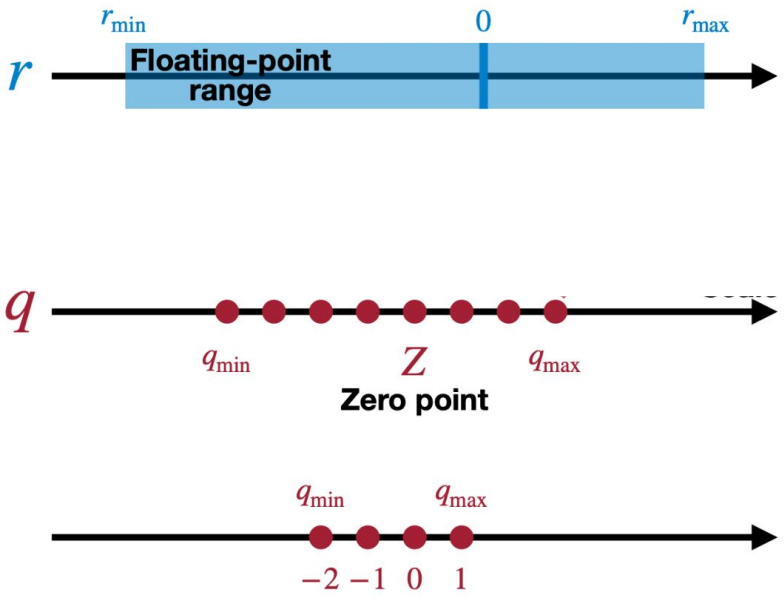


2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*

Binary	Decimal
01	1
00	0
11	-1
10	-2

Everything's ready for scale computation

Jacob et al., CVPR 2018

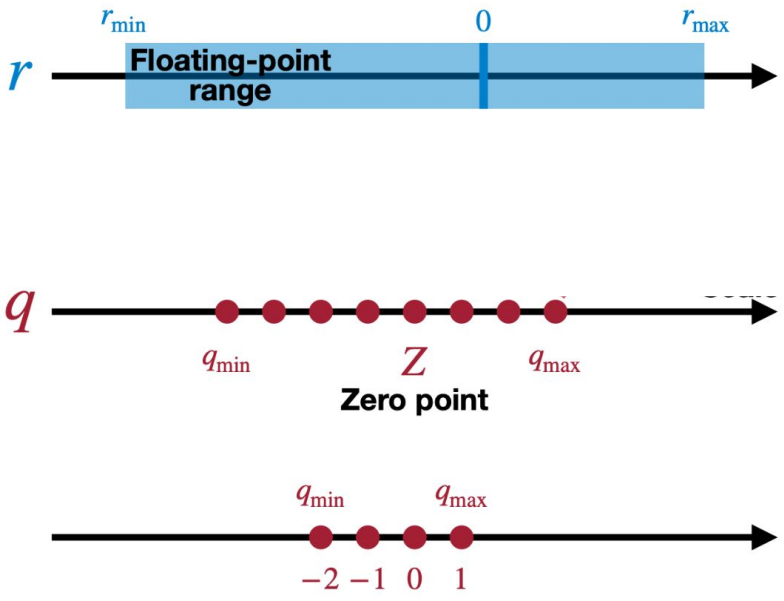


2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

Binary	Decimal
01	1
00	0
11	-1
10	-2

Jacob et al., CVPR 2018



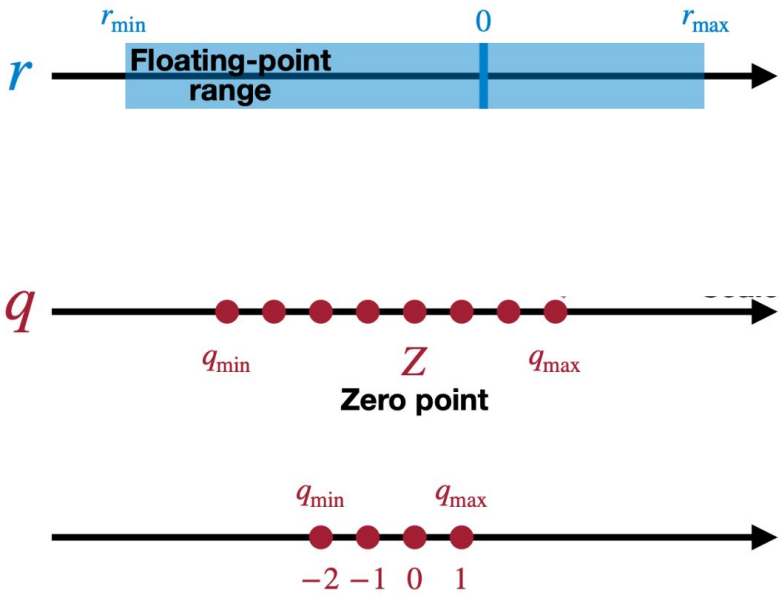
2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

Binary	Decimal
01	1
00	0
11	-1
10	-2

With actual numbers…

Jacob et al., CVPR 2018



2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

Binary	Decimal
01	1
00	0
11	-1
10	-2

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

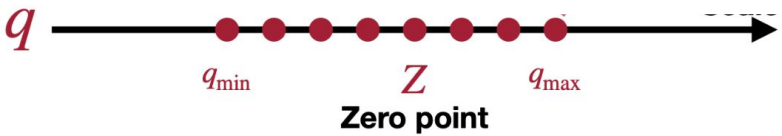
$$= \frac{2.12 - (-1.08)}{1 - (-2)}$$

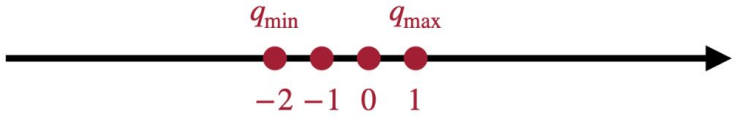
$$= 1.07$$

Jacob et al., CVPR 2018

2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

Got the scale, now need the Z!





Binary	Decimal
01	1
00	0
11	-1
10	-2

$$S = \frac{r_{max} - r_{min}}{q_{max} - q_{min}}$$

$$= \frac{2.12 - (-1.08)}{1 - (-2)}$$

$$= 1.07$$

Jacob et al., CVPR 2018

How to quantize weights?

$$q = \text{clip}\left(\text{round}\left(\frac{r}{S} + Z\right),\ q_{\text{min}},\ q_{\text{max}}\right)$$

r → float value ✅

q → integer-quantized value 🪄

S → scale ✅

Z → zero point 🤷

$q_{\{min,max\}}$ → integer dtype range ✅

Z → zero point 🤷

$$Z = \text{round}\left(q_{\text{min}} - \frac{r_{\text{min}}}{S}\right)$$

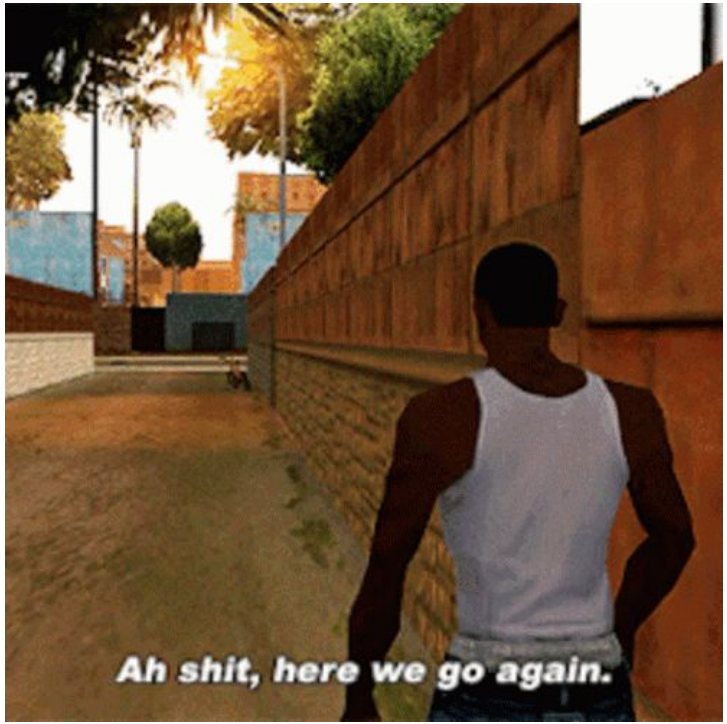
$r_{\{min,max\}}$ → original float value range ✅

$$q_{\text{min}} = -2, \quad q_{\text{max}} = 1$$

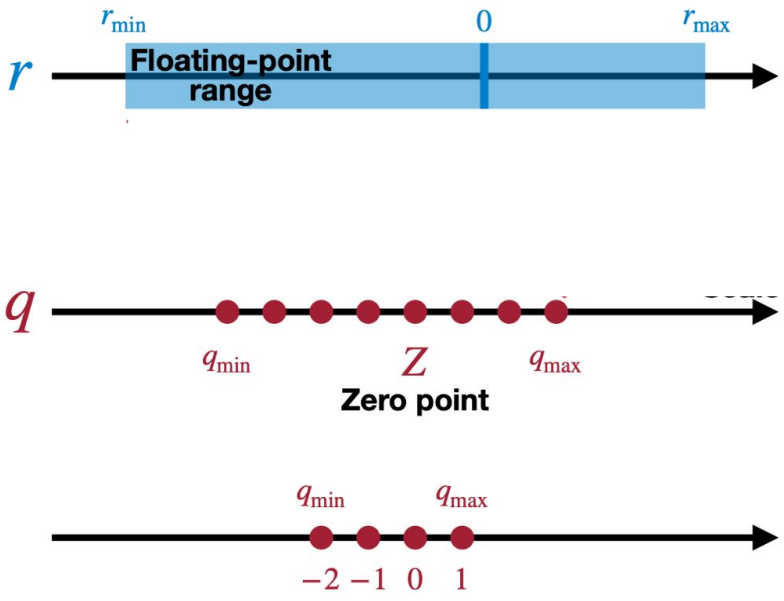
$$r_{\text{min}} = -1.08, \quad r_{\text{max}} = 2.12$$

$$Z = \text{round}\left(-2 - \frac{-1.08}{1.07}\right) = \text{round}(-0.99) \approx -1$$

Jacob et al., CVPR 2018



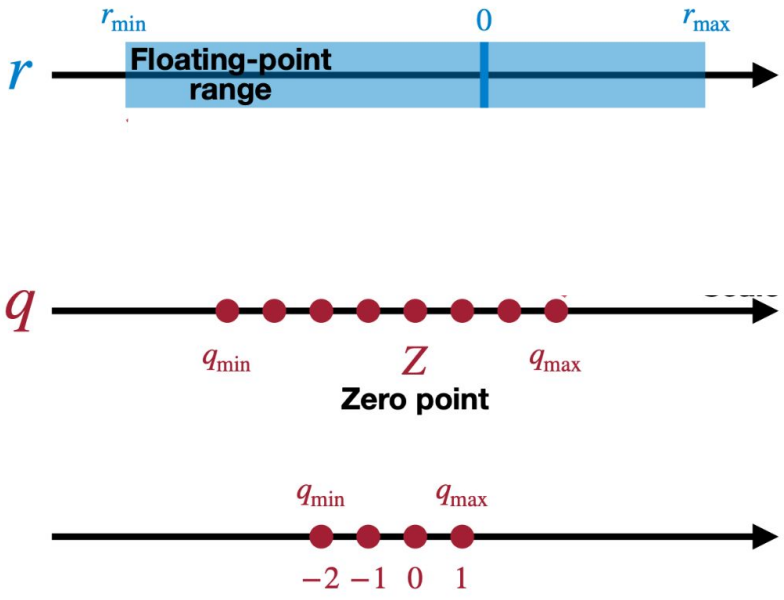
Jacob et al., CVPR 2018



2.09	*-0.98*	*1.48*	*0.09*
0.05	*-0.14*	*-1.08*	*2.12*
-0.91	*1.92*	*0*	*-1.03*
1.87	*0*	*1.53*	*1.49*

Binary	Decimal
01	1
00	0
11	-1
10	-2

Jacob et al., CVPR 2018

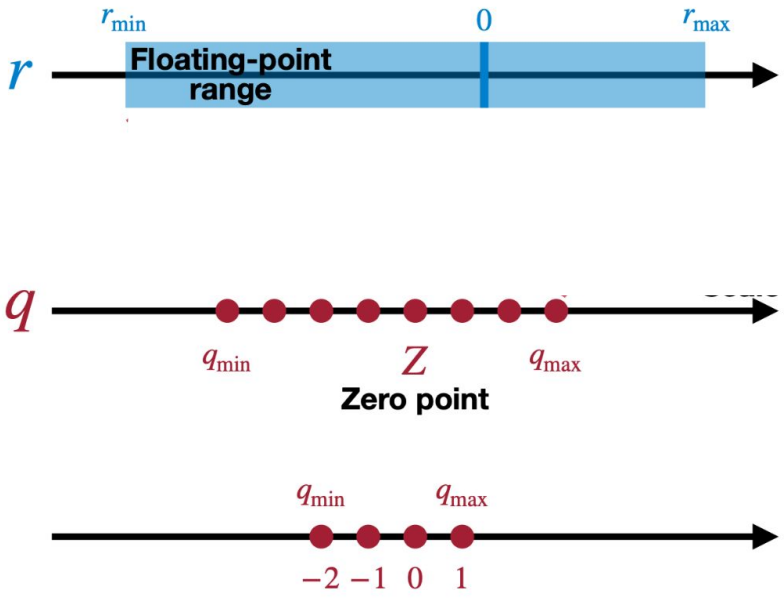


2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

$$Z = q_{\min} - \frac{r_{\min}}{S}$$

Binary	Decimal
01	1
00	0
11	-1
10	-2

Jacob et al., CVPR 2018



2.09	-0.98	1.48	0.09
0.05	-0.14	-1.08	2.12
-0.91	1.92	0	-1.03
1.87	0	1.53	1.49

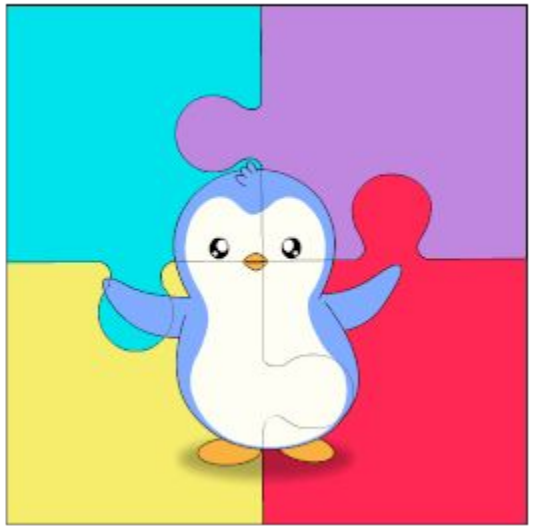
Binary	Decimal
01	1
00	0
11	-1
10	-2

$$Z = q_{\min} - \frac{r_{\min}}{S}$$

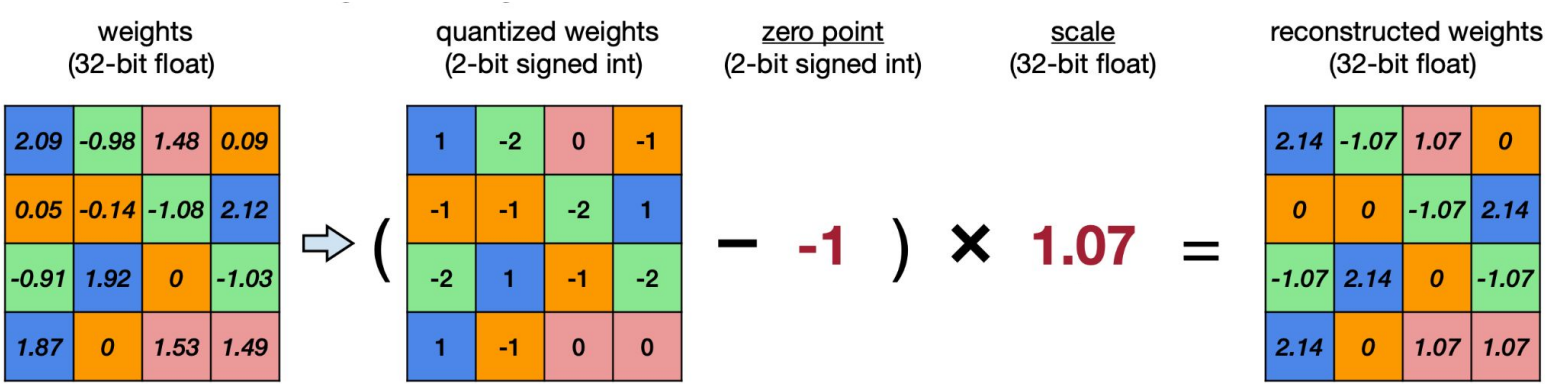
$$= \text{round}(-2 - \frac{-1.08}{1.07})$$

$$= -1$$

Jacob et al., CVPR 2018



Jacob et al., CVPR 2018



Binary	Decimal
01	1
00	0
11	-1
10	-2

Jacob et al., CVPR 2018

Jacob et al., CVPR 2018

We know how to quantize a float to int…

Jacob et al., CVPR 2018

We know how to quantize a float to int…

… and we also know how to dequantize the int back to float…

Jacob et al., CVPR 2018

We know how to quantize a float to int…

… and we also know how to dequantize the int back to float…

But we need to know how to do the damned mat muls now!

Jacob et al., CVPR 2018

We know how to quantize a float to int…

… and we also know how to dequantize the int back to float…

But we need to know how to do the damned mat muls now!

Well, not now! Next week!

Jacob et al., CVPR 2018

We know how to quantize a float to int…

… and we also know how to dequantize the int back to float…

But we need to know how to do the damned mat muls now!

Well, not now! Next week!

Now we consolidate with labs on stuff seen up until now!